Chapter 6 - Classification - Basic Concepts and Methods

J. Han, J. Pei and H. Tong, *Data Mining, Concepts and Techniques*, 4th ed., Morgan Kaufmann, 2023.

Contents

- 1. Basic Concepts
- 2. Decision Tree Induction
- 3. Bayes Classification Methods
- 4. Lazy Learners
- 5. Linear Classifiers
- 6. Model Evaluation and Selection
- 7. Techniques to Improve Classification Accuracy

1. Basic Concepts
1.1. What Is Classification?

- Classification is a form of data analysis that extracts models describing important data classes.
  - Such models (called classifiers) predict categorical (discrete, unordered) class labels.

1. Basic Concepts
1.2. General Approach to Classification

- "*How does classification work?*"
- Classification is a two-step process, consisting of
  - 1. A *learning step*: build a classification model based on data
  - 2. A *classification step*: if the model's accuracy is acceptable, it is used to predict class labels for given data

1. Basic Concepts
1.2. General Approach to Classification

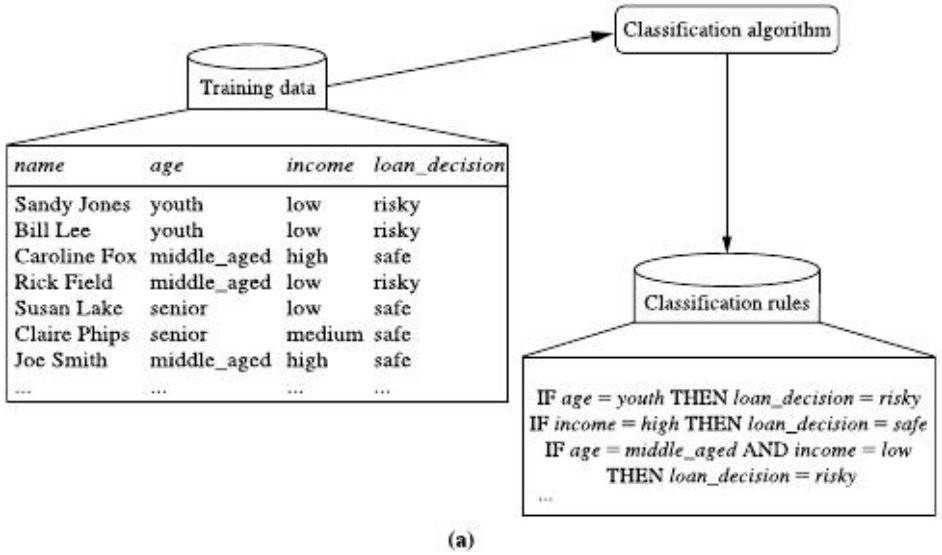


FIGURE 6.1

The data classification process: (a) *Learning*: Training data are analyzed by a classification algorithm. Here, the class label attribute is *loan_decision*, and the learned model or classifier is represented in the form of classification rules. (b) *Classification*: Test data are used to estimate the accuracy of the classification rules. If the accuracy is acceptable, the rules can be applied to the classification of new data tuples.

1. Basic Concepts
1.2. General Approach to Classification

- Learning Step (also known as the training phase):
  - A classification algorithm builds the classifier by analyzing or "learning from" a **training set** made up of database tuples and their associated class labels.
  - A tuple, $\boldsymbol{X}$, is represented by an n-dimensional attribute vector, $\boldsymbol{X} = (x_1, x_2, ..., x_n)$,
    - Depicting n measurements made on the tuple from n database attributes, respectively, $A_1, A_2, ..., A_n$.
  - Each tuple, $\boldsymbol{X}$, is assumed to belong to a predefined class as determined by another database attribute called the **class label attribute**.
    - The class label attribute is discrete-valued and unordered. It is categorical (or nominal) in that each value serves as a category or class.
    - The individual tuples making up the **training set** are referred to as training tuples and are randomly sampled from the database under analysis.
    - In the context of classification, data tuples can be referred to as *samples*, *examples*, *instances*, *data points*, or *objects*.

1. Basic Concepts
1.2. General Approach to Classification

- Supervised Learning
  - The class label of each training tuple is provided.
  - The learning of the classifier is "supervised" in that it is told to which class each training tuple belongs.
  - The scope of supervised learning is larger than classification, and it broadly encompasses learning methods for training a numerical prediction model (e.g., regression, ranking) if the true target values of training tuples are known during the learning step.
- Unsupervised Learning (Clustering)
  - Contrasts with supervised learning: The true target value (e.g., class label) of each training tuple is not known, and the number or set of classes to be learned may not be known in advance.
  - For example, if we did not have the *loan_decision* data available for the training set, we could use clustering to try to determine "groups of like tuples" which may correspond to risk groups within the loan application data.
  - Likewise, we could use clustering techniques to find social media posts sharing similar topics without knowing their actual class labels.

1. Basic Concepts
1.2. General Approach to Classification

- Weakly Supervised Learning
  - Semisupervised Learning
    - It builds a classifier based on a limited number of labeled training tuples (whose true class labels are given during training) and a large number of unlabeled training tuples (whose class labels are unknown during training);
  - Zero-shot Learning
    - Some class label might appear after the classification model has been built. In other words, during the training phase, there are no (i.e., zero) labeled training tuples for such a class label.

1. Basic Concepts
1.2. General Approach to Classification

- Learning step of the classification process can also be viewed as the learning of a mapping or function, $y = f(\boldsymbol{X})$, that can predict the associated class label y of a given tuple $\boldsymbol{X}$.
  - In this view, we wish to learn a mapping or function that separates the data classes. Typically, this mapping is represented in the form of classification rules, decision trees, or mathematical formulae.

1. Basic Concepts
1.2. General Approach to Classification

- "*What about classification accuracy?*"
  - In the second step, the model is used for classification.
  - First, the predictive accuracy of the classifier is estimated.
    - If we were to use the training set to measure the classifier's accuracy, this estimate would likely be too optimistic, because the classifier tends to **overfit** the data.
    - Therefore a **test set** is used, made up of **test tuples** and their associated class labels. They are independent of the training tuples, meaning that they were not used to construct the classifier.
    - The **accuracy** of a classifier on a given test set is the percentage of test tuples that are correctly classified by the classifier. The associated class label of each test tuple is compared with the learned classifier's class prediction for that tuple.
  - If the accuracy of the classifier is considered acceptable, the classifier can be used to classify future data tuples for which the class label is not known.

2. Decision Tree Induction

- **ID3, C4.5, CART**
- **Decision tree induction** is the learning of decision trees from class-labeled training tuples.
- A **decision tree** is a flowchart-like tree structure, where
  - Each **internal node** (nonleaf node) denotes a test on an attribute.
  - Each **branch** represents an outcome of the test.
  - Each **leaf node** (or *terminal node*) holds a class label.
  - The topmost node in a tree is the **root** node.

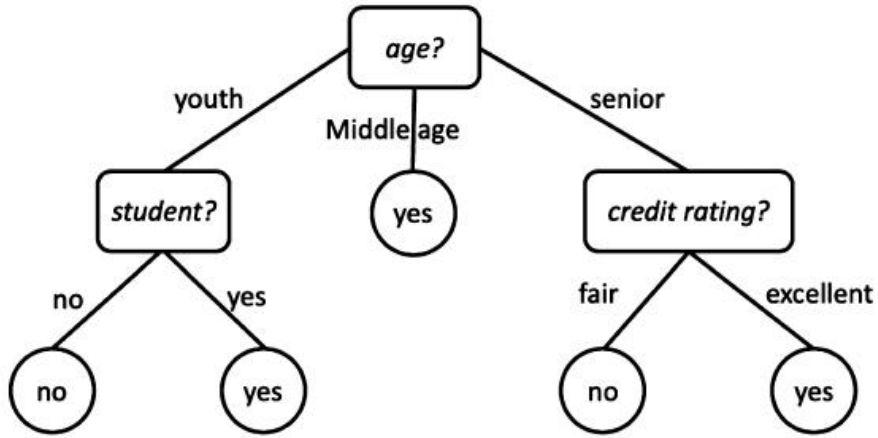


FIGURE 6.2

A decision tree for the concept *buys_computer*, indicating whether a customer is likely to purchase a computer. Each internal (nonleaf) node represents a test on an attribute. Each leaf node represents a class (either *buys_computer* = *yes* or *buys_computer* = *no*).

2. Decision Tree Induction

- "*How are decision trees used for classification?*"
  - Given a tuple, ***X***, for which the associated class label is unknown, the attribute values of the tuple are tested against the decision tree.
  - A path is traced from the root to a leaf node, which holds the class prediction for that tuple.
  - Decision trees can easily be converted to classification rules.
- "*Why are decision tree classifiers so popular?*"
  - The construction of decision tree classifiers does not require any domain knowledge or parameter setting and therefore is appropriate for exploratory knowledge discovery.
  - The learning and classification steps of decision tree induction are simple and fast. In general, decision tree classifiers have good accuracy. However, successful use may depend on the data at hand.
  - Decision tree induction algorithms have been used for classification in many application areas such as medicine, manufacturing and production, financial analysis, astronomy, and molecular biology. Decision trees are the basis of several commercial rule induction systems.

2. Decision Tree Induction
2.1. Decision Tree Generation

Algorithm: Generate_decision_tree. Generate a decision tree from the training tuples of data partition, D.

Input:

- Data partition, D, which is a set of training tuples and their associated class labels;
- *attribute_list*, the set of candidate attributes;
- *Attribute_selection_method*, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a *splitting_attribute* and, possibly, either a *split-point* or *splitting subset*.

Output: A decision tree.

Method:

```
(1)  create a node N;
(2)  if tuples in D are all of the same class, C, then
(3)      return N as a leaf node labeled with the class C;
(4)  if attribute_list is empty then
(5)      return N as a leaf node labeled with the majority class in D; // majority voting
(6)  apply Attribute_selection_method(D, attribute_list) to find the "best" splitting_criterion;
(7)  label node N with splitting_criterion;
(8)  if splitting_attribute is discrete-valued and
         multiway splits allowed then // not restricted to binary trees
(9)      attribute_list ← attribute_list − splitting_attribute; // remove splitting_attribute
(10) for each outcome j of splitting_criterion
     // partition the tuples and grow subtrees for each partition
(11)     let Dj be the set of data tuples in D satisfying outcome j; // a partition
(12)     if Dj is empty then
(13)         attach a leaf labeled with the majority class in D to node N;
(14)     else attach the node returned by Generate_decision_tree(Dj, attribute_list) to node N;
     endfor
(15) return N.
```

FIGURE 6.3

Basic algorithm for inducing a decision tree from training tuples.

- The algorithm is called with three parameters: *D*, *attribute_list*, and *Attribute_selection_method*.
  - D is a data partition. Initially, it is the complete set of training tuples and their associated class labels.
  - The parameter *attribute_list* is a list of attributes describing the tuples.
  - *Attribute_selection_method* specifies a heuristic procedure for selecting the attribute that "best" discriminates the given tuples according to class.

2. Decision Tree Induction
2.1. Decision Tree Generation

- *Attribute_selection_method* to determine the **splitting criterion**.
  - The splitting criterion tells us which attribute to test at node *N* by determining the "best" way to separate or partition the tuples in *D* into individual classes.
  - The splitting criterion also tells us which branches to grow from node *N* with respect to the outcomes of the chosen test. More specifically, the splitting criterion indicates the splitting attribute and may also indicate either a **split-point** or a **splitting subset**.
  - The splitting criterion is determined so that, ideally, the resulting partitions at each branch are as "pure" as possible.
    - A partition is **pure** if all the tuples in it belong to the same class.
    - In other words, if we split up the tuples in *D* according to the mutually exclusive outcomes of the splitting criterion, we hope for the resulting partitions to be as pure as possible.

2. Decision Tree Induction
2.1. Decision Tree Generation

- The node *N* is labeled with the splitting criterion, which serves as a test at the node. A branch is grown from node *N* for each of the outcomes of the splitting criterion. The tuples in D are partitioned accordingly.
- Let *A* be the splitting attribute. *A* has *v* distinct values, $\{a_1, a_2, ..., a_v\}$, based on the training data.
- There are three possible scenarios:

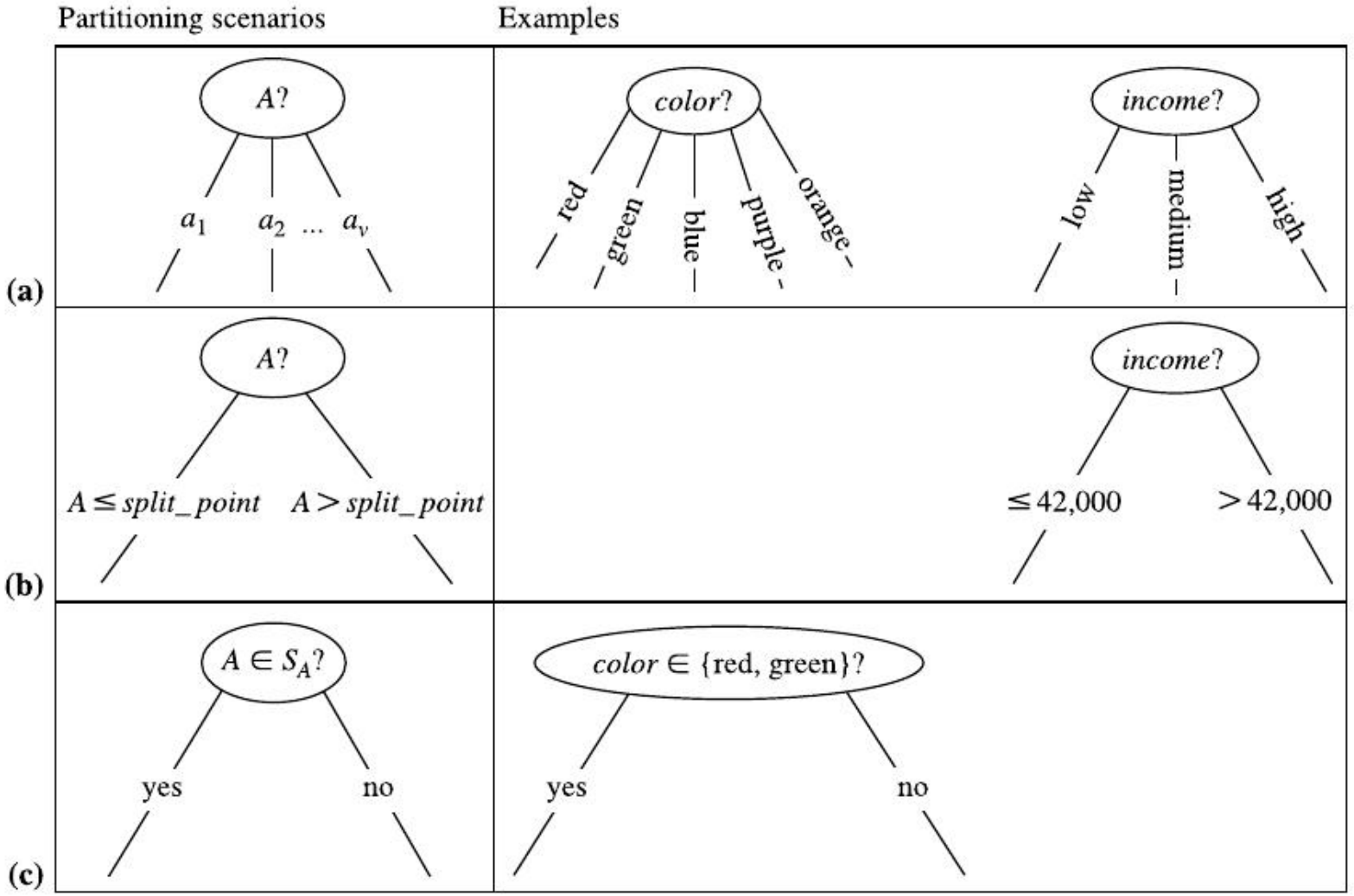


FIGURE 6.4

This figure shows three possibilities for partitioning tuples based on the splitting criterion, each with examples. Let A be the splitting attribute. (a) If A is discrete-valued, then one branch is grown for each known value of A. (b) If A is continuous-valued, then two branches are grown, corresponding to $A \leq split\_point$ and $A > split\_point$. (c) If A is discrete-valued and a binary tree must be produced, then the test is of the form $A \in S_A$, where S_A is the splitting subset for A.

2. Decision Tree Induction
2.1. Decision Tree Generation

- The computational complexity of the algorithm given training set D is $O(n \times |D| \times log(|D|))$, where n is the number of attributes describing the tuples in D and $|D|$ is the number of training tuples in D.

2. Decision Tree Induction
2.2. Attribute Selection Measures

- An **attribute selection measure** is a heuristic for selecting the splitting criterion that "best" separates a given data partition, *D*, of class-labeled training tuples into individual classes.
- If we were to split *D* into smaller partitions according to the outcomes of the splitting criterion, ideally, each partition would be pure (i.e., all the tuples that fall into a given partition would belong to the same class).
- Conceptually, the "best" splitting criterion is the one that most closely results in such a scenario.
- Attribute selection measures are also known as **splitting rules** because they determine how the tuples at a given node are to be split.

2. Decision Tree Induction
2.2. Attribute Selection Measures

- The attribute selection measure provides a ranking for each attribute describing the given training tuples. The attribute having the best score for the measure is chosen as the splitting attribute for the given tuples.
- If the splitting attribute is continuous-valued or if we are restricted to binary trees, then, respectively, either a *split point* or a *splitting subset* must also be determined as part of the splitting criterion.
- The tree node created for partition D is labeled with the splitting criterion, branches are grown for each outcome of the criterion, and the tuples are partitioned accordingly.
- This section describes three popular attribute selection measures—*information gain*, *gain ratio*, and *Gini impurity*.
- Let D, the data partition, be a training set of class-labeled tuples. Suppose the class label attribute has m distinct values defining m **distinct classes**, C_i (for $i = 1, ..., m$). Let $C_{i,D}$ be the set of tuples of class C_i in D. Let $|D|$ and $|C_{i,D}|$ denote the number of tuples in D and $C_{i,D}$, respectively.

2. Decision Tree Induction

2.2. Attribute Selection Measures

Information Gain

- ID3 uses **information gain** as its attribute selection measure.
- This measure is based on pioneering work by Claude Shannon on information theory, which studied the value or "information content" of messages.
- Let node *N* represent the tuples of partition *D*. The attribute with the *highest information gain* is chosen as the splitting attribute for node *N*. This attribute minimizes the information needed to classify the tuples in the resulting partitions and reflects the least randomness or "impurity" in these partitions.
- Such an approach minimizes the expected number of tests needed to classify a given tuple and guarantees that a simple (but not necessarily the simplest) tree is found.

2. Decision Tree Induction
2.2. Attribute Selection Measures
Information Gain

- The expected information needed to classify a tuple in D is given by

$$Info(D) = -\sum_{i=1}^{m} p_i log_2(p_i)$$

where p_i is the nonzero probability that an arbitrary tuple in D belongs to class C_i and is estimated by $|C_{i,D}|/|D|$.

  - A log function to the base 2 is used, because the information is encoded in bits.
  - *Info*(D) is just the average amount of information needed to identify the class label of a tuple in D. Note that, at this point, the information we have is based solely on the proportions of tuples of each class.
  - *Info*(D) is also known as the **entropy** of D.

2. Decision Tree Induction
2.2. Attribute Selection Measures
Information Gain

- Now, suppose we were to partition the tuples in D on some attribute A having v distinct values, $\{a_1, a_2, ..., a_v\}$, as observed from the training data.
  - If A is discrete-valued, these values correspond directly to the v outcomes of a test on A. Attribute A can be used to split D into v partitions or subsets, $\{D_1, D_2, ..., D_v\}$, where D_j contains those tuples in D that have outcome a_j of A.
  - These partitions would correspond to the branches grown from node N. Ideally, we would like this partitioning to produce an exact classification of the tuples. That is, we would like for each partition to be pure.
  - However, it is quite likely that the partitions will be impure (e.g., where a partition may contain a collection of tuples from different classes rather than from a single class).

2. Decision Tree Induction
2.2. Attribute Selection Measures
Information Gain

- How much more information would we still need (after the partitioning) to arrive at an exact classification? This amount is measured by

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j) \qquad (6.3)$$

  - The term $\frac{|D_j|}{|D|}$ acts as the weight of the jth partition.
  - $Info_A(D)$ is the expected information required to classify a tuple from D based on the partitioning by A.
    - The smaller the expected information (still) required, the greater the purity of the partitions.
    - $Info_A(D)$ is also known as the conditional entropy of D (conditioned on the attribute A).

2. Decision Tree Induction

2.2. Attribute Selection Measures

Information Gain

- Information gain is defined as the difference between the original information requirement (i.e., based on just the proportion of classes) and the new requirement (i.e., obtained after partitioning on A).

$$Gain(A) = Info(D) - Info_A(D)$$

  - In other words, $Gain(A)$ tells us how much would be gained by branching on A. It is the expected reduction in the information requirement caused by knowing the value of A.

- The attribute A with the highest information gain, $Gain(A)$, is chosen as the splitting attribute at node N.

2. Decision Tree Induction
2.2. Attribute Selection Measures
Information Gain

- **Example 6.1. Induction of a decision tree using information gain.** Table 6.1 presents a training set, *D*, of class-labeled tuples randomly selected from the customer database of an electronics store.
  - In this example, each attribute is discrete-valued.
  - The class label attribute, *buys_computer*, has two distinct values (namely, {*yes*, *no*}); therefore, there are two distinct classes (i.e., $m = 2$).
  - Let class C_1 correspond to *yes* and class C_2 correspond to *no*.
  - There are nine tuples of class *yes* and five tuples of class *no*.
  - A (root) node *N* is created for the tuples in *D*.

Table 6.1 Class-labeled training tuples from the customer database of an electronics store.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

2. Decision Tree Induction

2.2. Attribute Selection Measures

Information Gain

- The expected information needed to classify a tuple in D:

$$Info(D) = -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right) = 0.940 \text{ bits.}$$

- The expected information needed to classify a tuple in D if the tuples are partitioned according to *age* is

$$\begin{aligned} Info_{age}(D) = {} & \frac{5}{14} \times \left(-\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5}\right) \\ & + \frac{4}{14} \times \left(-\frac{4}{4}\log_2\frac{4}{4}\right) \\ & + \frac{5}{14} \times \left(-\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5}\right) \\ & = 0.694 \text{ bits.} \end{aligned}$$

Table 6.1 Class-labeled training tuples from the customer database of an electronics store.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

- Hence, the gain in information from such partitioning would be

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

2. Decision Tree Induction
2.2. Attribute Selection Measures
Information Gain

- Similarly, we can compute
  - $Gain(income) = 0.029$ bits
  - $Gain(student) = 0.151$ bits
  - $Gain(credit\_rating) = 0.048$ bits.
- Because *age* has the highest information gain among the attributes, it is selected as the splitting attribute. Node *N* is labeled with *age*, and branches are grown for each of the attribute's values.

2. Decision Tree Induction

2.2. Attribute Selection Measures

Information Gain

Table 6.1 Class-labeled training tuples from the custom an electronics store.

RID	age	income	student	credit_rating	Class:
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

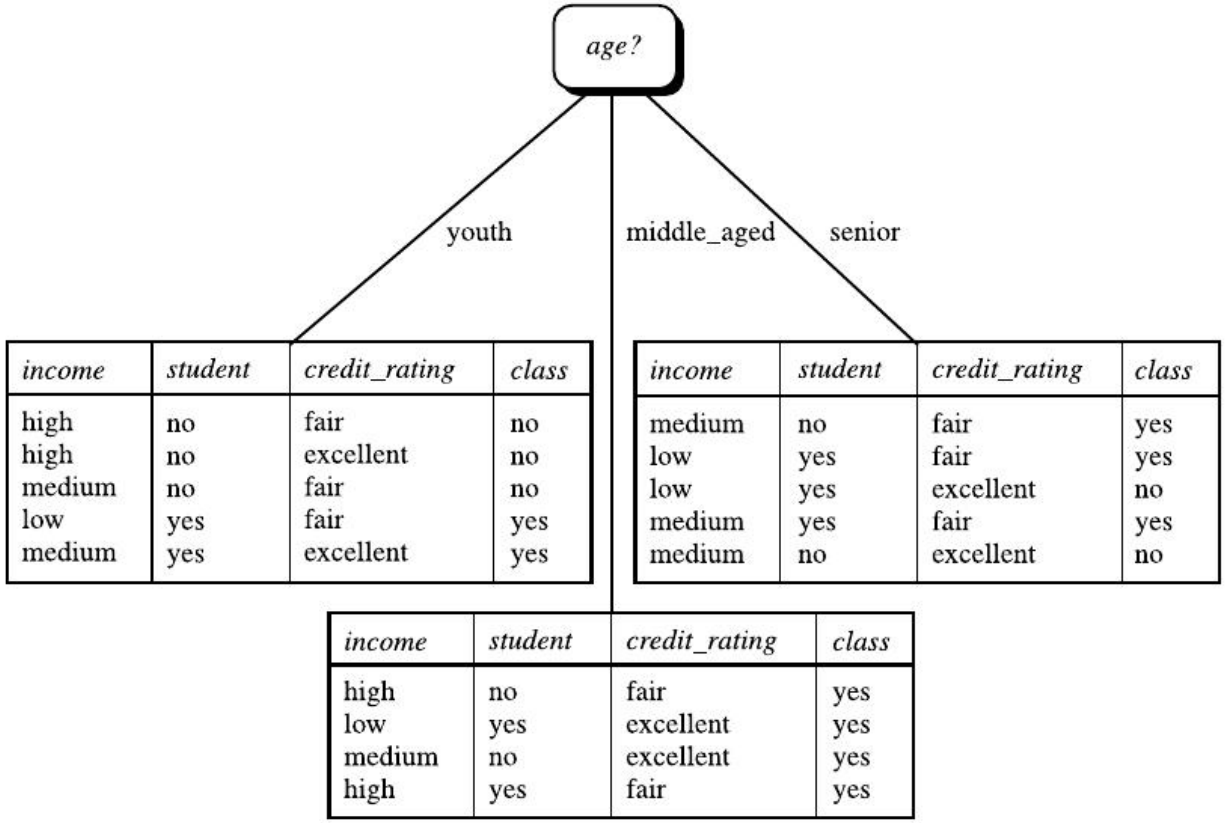


FIGURE 6.6

The attribute *age* has the highest information gain and therefore becomes the splitting attribute at the root node of the decision tree. Branches are grown for each outcome of *age*. The tuples are shown partitioned accordingly.

2. Decision Tree Induction
2.2. Attribute Selection Measures
Information Gain

- "*But how can we compute the information gain of an attribute that is continuous-valued, unlike in the example?*"
- Suppose, instead, that we have an attribute A that is continuous-valued rather than discrete-valued. For such a scenario, we must determine the "best" **split-point** for A, where the split-point is a threshold on A.
- We first sort the values of A in the increasing order. Typically, the midpoint between each pair of adjacent values is considered as a possible split-point.
- Therefore, given v values of A, $(v - 1)$ possible splits are evaluated.
  - For example, the midpoint between the values a_i and a_{i+1} of A is $(a_i + a_{i+1}) / 2$.
- For each possible split-point for A, we evaluate $\text{Info}_A(D)$, where the number of partitions is two, that is, $v = 2$ in Eq. (6.3). The point with the minimum expected information requirement for A is selected as the *split_point* for A. D_1 is the set of tuples in D satisfying $A \leq split\_point$, and D_2 is the set of tuples in D satisfying $A > split\_point$.

2. Decision Tree Induction
2.2. Attribute Selection Measures
Gain Ratio

- The information gain measure is biased toward tests with many outcomes. That is, it prefers to select attributes having a large number of values.
  - For example, consider an attribute that acts as a unique identifier, such as *product_ID*. A split on *product_ID* would result in a large number of partitions (as many as there are values), each one containing just one tuple.
  - Because each partition is pure, the information required to classify data set *D* based on this partitioning would be $Info_{product\_ID}(D) = 0$. Therefore the information gained by partitioning on this attribute is maximal. Clearly, such a partitioning is useless for classification.
- C4.5, a successor of ID3, uses an extension to information gain known as *gain ratio*, which attempts to overcome this bias. It applies a kind of normalization to information gain using a "split information" value defined analogously with *Info*(*D*) as

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times log_2\left(\frac{|D_j|}{|D|}\right) \quad (6.6)$$

2. Decision Tree Induction
2.2. Attribute Selection Measures
Gain Ratio

$$SplitInfo_A(D) = -\sum_{j=1}^{v} \frac{|D_j|}{|D|} \times log2\left(\frac{|D_j|}{|D|}\right)$$

- This value represents the potential information generated by splitting the training data set, D, into v partitions, corresponding to the v outcomes of a test on attribute A.
- Note that, for each outcome, it considers the number of tuples having that outcome with respect to the total number of tuples in D.
- It differs from information gain, which measures the information with respect to classification that is acquired based on the same partitioning.

- The gain ratio is defined as

$$GainRatio(A) = \frac{Gain(A)}{SplitInfo_A(D)}$$

- The attribute with the maximum gain ratio is selected as the splitting attribute.
- Note, however, that as the split information approaches 0, the ratio becomes unstable. A constraint is added to avoid this, whereby the information gain of the test selected must be large—at least as great as the average gain over all tests examined.

2. Decision Tree Induction
2.2. Attribute Selection Measures
Gain Ratio

- **Example 6.2. Computation of gain ratio for the attribute *income*.**
- A test on income splits the data of Table 6.1 into three partitions, namely *low*, *medium*, and *high*, containing four, six, and four tuples, respectively.

Table 6.1 Class-labeled training tuples from the customer database of an electronics store.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

$$SplitInfo_{income}(D) = -\frac{4}{14} \times \log_2\left(\frac{4}{14}\right) - \frac{6}{14} \times \log_2\left(\frac{6}{14}\right) - \frac{4}{14} \times \log_2\left(\frac{4}{14}\right)$$
$$= 1.557.$$

2. Decision Tree Induction

2.2. Attribute Selection Measures

Gain Ratio

- From Example 6.1, we have *Gain*(*income*) = 0.029. Therefore *GainRatio*(*income*) = 0.029/1.557 = 0.019.

2. Decision Tree Induction
2.2. Attribute Selection Measures
Gini Impurity

- The Gini impurity (or Gini in short) is used in CART.
- The Gini measures the impurity of D, a data partition or a set of training tuples, as

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2 \quad (6.8)$$

  - where p_i is the probability that a tuple in D belongs to class C_i and is estimated by $|C_{i,D}|/|D|$. The sum is computed over m classes.
- The Gini impurity considers a binary split for each attribute.

2. Decision Tree Induction
2.2. Attribute Selection Measures
Gini Impurity

- Let's first consider the case where A is a discrete-valued attribute having v distinct values, $\{a_1, a_2, ..., a_v\}$, occurring in D.
- To determine the best binary split on A, we examine all the possible subsets that can be formed using known values of A. Each subset, S_A, can be considered as a binary test for attribute A of the form "$A \in S_A$?"
  - Given a tuple, this test is satisfied if the value of A for the tuple is among the values listed in S_A.
- If A has v possible values, then there are 2^v possible subsets.
  - For example, if *income* has three possible values, namely {*low, medium, high*}, then the possible subsets are {*low, medium, high*}, {*low, medium*}, {*low, high*}, {*medium, high*}, {*low*}, {*medium*}, {*high*}, and {}.
  - We exclude the power set, {low, medium, high}, and the empty set from consideration since, conceptually, they do not represent a split.
  - Therefore there are $(2^v - 2)/2$ possible ways to form two partitions of the data, D, based on a binary split on A.

2. Decision Tree Induction

2.2. Attribute Selection Measures

Gini Impurity

- When considering a binary split, we compute a weighted sum of the impurity of each resulting partition.
- For example, if a binary split on A partitions D into D_1 and D_2, the Gini impurity of D given that partitioning is

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

- For each attribute, each of the possible binary splits is considered.

Gini Impurity

- For a discrete-valued attribute, the subset that gives the minimum Gini impurity for that attribute is selected as its splitting subset.
- For continuous-valued attributes, each possible split-point must be considered.
  - The strategy is similar to that described earlier for information gain, where the midpoint between each pair of (sorted) adjacent values is taken as a possible split-point.
  - The point giving the minimum Gini impurity for a given (continuous-valued) attribute is taken as the split-point of that attribute. Recall that for a possible split-point of A, D_1 is the set of tuples in D satisfying $A \leq split\_point$, and D_2 is the set of tuples in D satisfying $A > split\_point$.

2. Decision Tree Induction

2.2. Attribute Selection Measures

Gini Impurity

- The reduction in impurity that would be incurred by a binary split on a discrete- or continuous-valued attribute A is

$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$

- The attribute that maximizes the reduction in impurity (or, equivalently, has the minimum Gini impurity) is selected as the splitting attribute.
- This attribute and either its splitting subset (for a discrete-valued splitting attribute) or split-point (for a continuous-valued splitting attribute) together form the splitting criterion.

2. Decision Tree Induction
2.2. Attribute Selection Measures
Gini Impurity

- **Example 6.3. Induction of a decision tree using the Gini impurity.** Let *D* be the training data shown earlier in Table 6.1, where there are nine tuples belonging to the class *buys_computer* = *yes* and the remaining five tuples belong to the class *buys_computer* = *no*.

- A (root) node *N* is created for the tuples in *D*. We first use Eq. (6.8) for the Gini impurity to compute the impurity of *D*:

$$Gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459.$$

Table 6.1 Class-labeled training tuples from the customer database of an electronics store.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

2. Decision Tree Induction

2.2. Attribute Selection Measures

Gini Impurity

- To find the splitting criterion for the tuples in D, we need to compute the Gini impurity for each attribute.
- Let's start with the attribute *income* and consider each of the possible splitting subsets.
  - Consider the subset $\{low, medium\}$. This would result in 10 tuples in partition D_1 satisfying the condition "$income \in \{low, medium\}$." The remaining four tuples of D would be assigned to partition D_2. The Gini impurity value computed based on this partitioning is

$$
\begin{aligned}
&Gini_{income \in \{low,medium\}}(D) \\
&\quad = \frac{10}{14} Gini(D_1) + \frac{4}{14} Gini(D_2) \\
&\quad = \frac{10}{14}\left(1 - \left(\frac{7}{10}\right)^2 - \left(\frac{3}{10}\right)^2\right) + \frac{4}{14}\left(1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2\right) \\
&\quad = 0.443 \\
&\quad = Gini_{income \in \{high\}}(D).
\end{aligned}
$$

2. Decision Tree Induction
2.2. Attribute Selection Measures
Gini Impurity

- Similarly, the Gini impurity values for splits on the remaining subsets are 0.458 (for the subsets {*low, high*} and {*medium*}) and 0.450 (for the subsets {*medium, high*} and {*low*}).
  - Therefore the best binary split for attribute income is on {*low, medium*} (or {high}) because it minimizes the Gini impurity.
- Evaluating *age*, we obtain {*youth, senior*} (or {*middle_aged*}) as the best split for age with a Gini impurity of 0.375.
- The attributes *student* and *credit_rating* are both binary, with Gini impurity values of 0.367 and 0.429, respectively.
- **The attribute *age* and splitting subset {*youth, senior*} therefore give the minimum Gini impurity overall, with a reduction in impurity of 0.459 − 0.357 = 0.102. The binary split "age ∈ {*youth, senior?*}" results in the maximum reduction in impurity of the tuples in *D* and is returned as the splitting criterion.**

2. Decision Tree Induction
2.3. Tree Pruning

- When a decision tree is built, many of the branches will reflect anomalies in the training data due to noise or outliers.
- Tree pruning methods address this problem of *overfitting* the data. Such methods typically use statistical measures to remove the least-reliable branches.

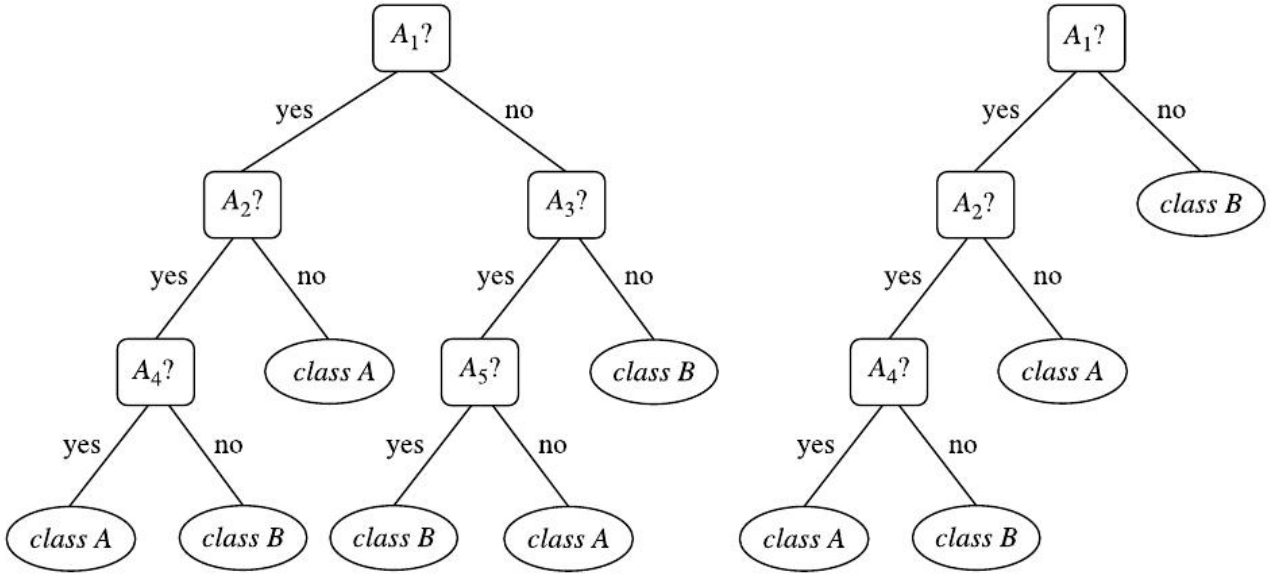


FIGURE 6.7

An unpruned decision tree (left) and a pruned version of it (right).

- Pruned trees tend to be smaller and less complex and, thus, easier to comprehend.
- They are usually faster and better at correctly classifying independent test data (i.e., of previously unseen tuples) than unpruned trees.
- "*How does tree pruning work?*"
  - There are two common approaches to tree pruning: *prepruning* and *postpruning*.

2. Decision Tree Induction

2.3. Tree Pruning

Prepruning

- In the **prepruning** approach, a tree is "pruned" by halting its construction early (e.g., by deciding not to further split or partition the subset of training tuples at a given node).
- Upon halting, the node becomes a leaf. The leaf may hold the most frequent class label among the subset tuples or the probability distribution of the class labels of those tuples.
- When constructing a tree, measures such as statistical significance, information gain, Gini impurity, and so on, can be used to assess the goodness of a split.
- If partitioning the tuples at a node would result in a split that falls below a prespecified threshold, then further partitioning of the given subset is halted.
  - There are difficulties, however, in choosing an appropriate threshold. High thresholds could result in oversimplified trees, whereas low thresholds could result in very little simplification.

2. Decision Tree Induction
2.3. Tree Pruning
Postpruning

- **Postpruning** removes subtrees from a "fully grown" tree.
- A subtree at a given node is pruned by removing its branches and replacing it with a leaf. The leaf is labeled with the most frequent class label among the subtree being replaced.
  - For example, notice the subtree at node "A3 ?" in the unpruned tree of Fig. 6.7. Suppose that the most common class within this subtree is "*class B.*" In the pruned version of the tree, the subtree in question is pruned by replacing it with the leaf "*class B.*"

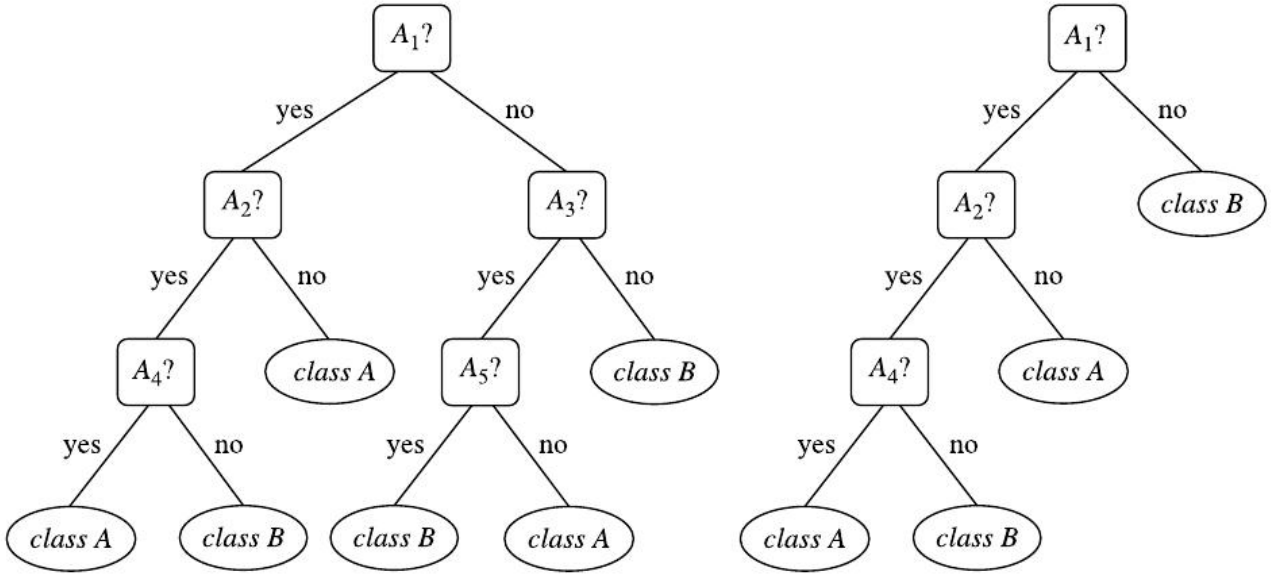


FIGURE 6.7

An unpruned decision tree (left) and a pruned version of it (right).

3. Bayes Classification Methods

- Bayesian classifiers are statistical classifiers.
  - They can predict class membership probabilities, such as the probability that a given tuple belongs to a particular class.
  - Bayesian classification is based on Bayes' theorem.
  - Studies comparing classification algorithms have found a simple Bayesian classifier known as the *naïve Bayesian classifier* to be comparable in performance with decision trees and selected neural network classifiers.
  - Bayesian classifiers have also exhibited high accuracy and speed when applied to large databases.
- Naïve Bayesian classifiers assume that the effect of an attribute value on a given class is independent of the values of the other attributes. This assumption is called *class-conditional independence*. It is made to simplify the computations involved and, in this sense, is considered "naïve."

3. Bayes Classification Methods

3.1. Bayes' Theorem

- Bayes' theorem is named after Thomas Bayes, a nonconformist English clergyman who did early work in probability and decision theory during the 18th century.
- Let $\boldsymbol{X}$ be a data tuple. In Bayesian terms, $\boldsymbol{X}$ is considered "evidence." As usual, it is described by measurements made on a set of n attributes.
- Let H be some hypothesis such as that the data tuple $\boldsymbol{X}$ belongs to a specified class C.
- For classification problems, we want to determine $P(H|\boldsymbol{X})$, the probability that the hypothesis H holds given the "evidence" or observed data tuple $\boldsymbol{X}$.
  - In other words, we are looking for the probability that tuple $\boldsymbol{X}$ belongs to class C, given that we know the attribute description of $\boldsymbol{X}$.

3. Bayes Classification Methods

3.1. Bayes' Theorem

- $P(H|\boldsymbol{X})$ is the **posterior probability**, or a *posteriori probability*, of H conditioned on $\boldsymbol{X}$.
  - For example, suppose our world of data tuples is confined to customers described by the attributes *age* and *income*, respectively, and that $\boldsymbol{X}$ is a 35-year-old customer with an income of $40,000.
  - Suppose that H is the hypothesis that our customer will buy a computer. Then $P(H|\boldsymbol{X})$ reflects the probability that customer $\boldsymbol{X}$ will buy a computer given that we know the customer's age and income.
- In contrast, $P(H)$ is the **prior probability**, or a *priori probability*, of H.
  - For our example, this is the probability that any given customer will buy a computer, regardless of age, income, or any other information, for that matter.
  - The posterior probability, $P(H|\boldsymbol{X})$, is based on more information (e.g., customer information) than the prior probability, $P(H)$, which is independent of $\boldsymbol{X}$.

3. Bayes Classification Methods

3.1. Bayes' Theorem

- Similarly, $P(\boldsymbol{X}|H)$ is the conditional probability of $\boldsymbol{X}$ conditioned on **H**. That is, it is the probability that a customer, $\boldsymbol{X}$, is 35 years old and earns $40,000, given that we know the customer will buy a computer.
  - In classification, $P(\boldsymbol{X}|H)$ is also often referred to as *likelihood*.
- $P(\boldsymbol{X})$ is the prior probability of $\boldsymbol{X}$. Using our example, it is the probability that a person from our set of customers is 35 years old and earns $40,000.
  - In classification, $P(\boldsymbol{X})$ is also often referred to as *marginal probability*.
- **Bayes' theorem** is useful in that it provides a way of calculating the posterior probability, $P(H|\boldsymbol{X})$, from $P(H)$, $P(\boldsymbol{X}|H)$, and $P(\boldsymbol{X})$. Bayes' theorem is

$$P(H|\boldsymbol{X}) = \frac{P(\boldsymbol{X}|H)P(H)}{P(\boldsymbol{X})}$$

3. Bayes Classification Methods

3.1. Bayes' Theorem

What does Bayes Classifier Look Like?

- Suppose that there are m classes, C_1, C_2, ..., C_m. Given a tuple, **X**, we want to predict which class it belongs to.
- In Bayes classifier, it first calculates the posterior probabilities for each of the m classes, $P(C_i|\boldsymbol{X})$ ($i = 1, ..., m$), and then predicts that tuple $\boldsymbol{X}$ belongs to the class with the **highest posterior probability**.
- In the above example, given a customer, $\boldsymbol{X}$, of 35 years old and earning \$40,000, we want to predict if the customer will buy a computer.
  - There are two possible classes (buy computer vs. not buy computer).
  - Suppose $P(\text{buy computer}|\boldsymbol{X}) = 0.8$ and $P(\text{not buy computer}|\boldsymbol{X}) = 0.2$. Bayes classifier will predict that the customer $\boldsymbol{X}$ will buy a computer.

3. Bayes Classification Methods

3.1. Bayes' Theorem

What does Bayes Classifier Look Like?

- According to Bayes' theorem

$$P(C_i|\boldsymbol{X}) = \frac{P(\boldsymbol{X}|C_i)P(C_i)}{P(\boldsymbol{X})}$$

  - In Bayes classifier, we only need to know which class has the highest posterior probability and for a given tuple $\boldsymbol{X}$, its marginal probability is independent of different classes.
  - In other words, different posterior probabilities $P(C_i|\boldsymbol{X})$ ($i = 1, ..., m$) share the same marginal probability $P(\boldsymbol{X})$.
  - Therefore for the purpose of predicting which class a given tuple belongs to, we only need to estimate the conditional probabilities $P(\boldsymbol{X}|C_i)$ ($i = 1, ..., m$), and the priors $P(C_i)$ ($i = 1, ..., m$).
    - It is relatively easy to estimate the priors $P(C_i)$ ($i = 1, ..., m$) from the training data set.
    - On the other hand, it is usually very challenging to directly estimate the conditional probabilities $P(\boldsymbol{X}|C_i)$ ($i = 1, ..., m$).

3. Bayes Classification Methods

3.1. Bayes' Theorem

What does Bayes Classifier Look Like?

- To see this, let us assume there are n binary attributes $A_1, A_2, ..., A_n$. Then, the n-dimensional attribute vector $\boldsymbol{X}$ has 2^n possible values and we need to estimate the conditional probability of each possible value of the attribute vector with respect to each class label.
- In other words, the attribute value space is exponential! It is very difficult to estimate such a large number of parameters for the conditional probabilities.
- Therefore the main difficulty for Bayes classifier lies in how to efficiently estimate the conditional probabilities, often with some approximation.
- Many solutions have been developed. One of such efforts, probably the simplest yet quite effective solution, is the naïve Bayesian classifier.

3. Bayes Classification Methods

3.1. Bayes' Theorem

How Good is Bayes Classifier?

- In theory, Bayes classifier is *optimal* in the sense that it has the smallest classification error rate compared to *all* other classifiers.
- Since Bayes classifier is a probabilistic method, it could make a wrong prediction for any given tuple.
  - In the above example, Bayes classifier predicts the customer will buy a computer. Since P(not buy computer|$\boldsymbol{X}$) = 0.2, there is 20% chance that the prediction the Bayes classifier makes is incorrect.
- However, since Bayes classifier always predicts the class with the maximum posterior probability, the probability that its prediction is wrong for a given tuple $\boldsymbol{X}$ (which is often called *risk*) is the lowest in comparison to all other classifiers.
  - Therefore, the overall classification error of Bayes classifier, which is the expectation (i.e., the weighted average) of the risk of all possible tuples, is the lowest in all possible classifiers.

3. Bayes Classification Methods

3.1. Bayes' Theorem

How Good is Bayes Classifier?

- Given its theoretic optimality, Bayes classifier plays a foundational role in the statistical machine learning community.
  - Many classifiers (e.g., naïve Bayesian classifier, *k*-Nearest-Neighbor classifier, logistic regression, Bayesian network, etc.) can be viewed as approximated Bayes classifiers.
- Bayes classifier is also useful in that it provides a theoretical justification for other classifiers that do not explicitly use Bayes' theorem.
  - Under certain assumptions, it can be shown that many neural network and curve-fitting algorithms output the maximum posteriori hypothesis, as does the Bayes classifier.

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- The **naïve Bayesian** classifier, or **simple Bayesian** classifier, follows the same procedure as Bayes classifier, except the way it estimates the conditional probabilities. In detail, it works as follows:

- 1. Let D be a training set of tuples and their associated class labels. As usual, each tuple is represented by an n-dimensional attribute vector, $\boldsymbol{X} = (x_1, x_2, ..., x_n)$, depicting n measurements made on the tuple from n attributes, respectively, $A_1, A_2, ..., A_n$.

- 2. Suppose that there are m classes, $C_1, C_2, ..., C_m$. Given a tuple, $\boldsymbol{X}$, the classifier will predict that $\boldsymbol{X}$ belongs to the class having the highest posterior probability, conditioned on $\boldsymbol{X}$. That is, the naïve Bayesian classifier predicts that tuple $\boldsymbol{X}$ belongs to the class C_i if and only if

$$P(C_i|\boldsymbol{X}) > P(C_j|\boldsymbol{X}) \quad \text{for } 1 \leq j \leq m, j \neq i.$$

Thus we maximize $P(C_i|\boldsymbol{X})$. The class C_i for which $P(C_i|\boldsymbol{X})$ is maximized is called the *maximum posteriori hypothesis*. By Bayes' theorem,

$$P(C_i|\boldsymbol{X}) = \frac{P(\boldsymbol{X}|C_i)P(C_i)}{P(\boldsymbol{X})}$$

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- 3. As $P(\boldsymbol{X})$ is constant for all classes, we only need to find out which class maximizes $P(\boldsymbol{X}|C_i)P(C_i)$.
  - If the class prior probabilities are not known, then it is commonly assumed that the classes are equally likely, that is, $P(C_1) = P(C_2) = \cdots = P(C_m)$, and we would therefore maximize $P(\boldsymbol{X}|C_i)$. Otherwise, we maximize $P(\boldsymbol{X}|C_i)P(C_i)$.
  - Note that the class prior probabilities may be estimated by $P(C_i) = |C_{i,D}|/|D|$, where $|C_{i,D}|$ is the number of training tuples of class C_i in D.
- 4. Given a data set with many attributes, it would be extremely computationally expensive to compute $P(\boldsymbol{X}|C_i)$ for the aforementioned reasons. To reduce computation in evaluating $P(\boldsymbol{X}|C_i)$, the naïve assumption of **class-conditional independence** is made. **This presumes that the attributes' values are conditionally independent of one another, given the class label of the tuple** (i.e., there are no dependence relationships among the attributes, *if* we know which class the tuple belongs to.). Thus

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- Thus

$$P(\boldsymbol{X}|C_i) = \prod_{k=1}^{n} P(x_k|C_i)$$
$$= P(x_1|C_i) \times P(x_2|C_i) \times \ldots \times P(x_n|C_i) \qquad (6.13)$$

- We can easily estimate the probabilities $P(x_1|C_i)$, $P(x_2|C_i)$, ..., $P(x_n|C_i)$ from the training tuples.
- Recall that here x_k refers to the value of attribute A_k for tuple $\boldsymbol{X}$. For each attribute, we look at whether the attribute is categorical or continuous-valued. For instance, to compute $P(\boldsymbol{X}|C_i)$, we consider the following:

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- To estimate $P(x_k|C_i)$
  - a. If A_k is categorical, then $P(x_1|C_i)$ is the number of tuples of class C_i in D having the value x_k for A_k, divided by $|C_{i,D}|$, the number of tuples of class C_i in D.
  - b. If A_k is continuous-valued, then we need to do a bit more work, but the calculation is pretty straightforward. A continuous-valued attribute is **typically assumed to have a Gaussian distribution** with a mean μ and standard deviation σ, defined by

$$g(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$P(x_k|C_i) = g(x_k, \mu_{C_i}, \sigma_{C_i})$$

    - We need to compute μ_{C_i} and σ_{C_i}, which are the mean (i.e., average) and standard deviation, respectively, of the values of attribute A_k for training tuples of class C_i.

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- 5. To predict the class label of $\boldsymbol{X}$, $P(\boldsymbol{X}|C_i)P(C_i)$ is evaluated for each class C_i. The classifier predicts that the class label of tuple $\boldsymbol{X}$ is the class C_i if and only if

$$P(\boldsymbol{X}|C_i)P(C_i) > P(\boldsymbol{X}|C_j)P(C_j) \text{ for } 1 \leq j \leq m, j \neq i.$$

In other words, the predicted class label is the class C_i for which $P(\boldsymbol{X}|C_i)P(C_i)$ is the maximum.

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- "*How effective is naïve Bayesian classifier?*"
- Notice that the only difference between naïve Bayesian classifier and Bayes classifier is the class-conditional independence assumption. Therefore if such an assumption indeed holds, naïve Bayesian classifier would be optimal with the smallest possible classification error.
- However, in practice this is not always the case, owing to inaccuracies in the assumptions made for its use, such as class-conditional independence, and the lack of available probability data. Nonetheless, various empirical studies of this classifier in comparison to decision trees and selected neural network classifiers have found it to be comparable in some domains. Another advantage of naïve Bayesian classifier is that it can naturally handle the missing attribute(s).

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- **Example 6.5. Predicting a class label using naïve Bayesian classification.**
  - Let C_1 correspond to the class *buys_computer* = *yes* and C_2 correspond to *buys_computer* = *no*. The tuple we wish to classify is
  - $\boldsymbol{X}$ = (*age* = *youth*, *income* = *medium*, *student* = *yes*, *credit_rating* = *fair*)
- We need to find out which class maximizes $P(\boldsymbol{X}|C_i)P(C_i)$, for $i = 1, 2$.

Table 6.1 Class-labeled training tuples from the customer database of an electronics store.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- The prior probalibity for each class $P(C_i)$, $i = 1, 2$:

  $P(buys\_computer = yes) = 9/14 = 0.643$

  $P(buys\_computer = no) = 5/14 = 0.357.$

- To compute $P(\boldsymbol{X}|C_i)$, for $i = 1, 2$, we compute the following conditional probabilities:

  $P(age = youth \mid buys\_computer = yes) = 2/9 = 0.222$

  $P(age = youth \mid buys\_computer = no) = 3/5 = 0.600$

  $P(income = medium \mid buys\_computer = yes) = 4/9 = 0.444$

  $P(income = medium \mid buys\_computer = no) = 2/5 = 0.400$

  $P(student = yes \mid buys\_computer = yes) = 6/9 = 0.667$

  $P(student = yes \mid buys\_computer = no) = 1/5 = 0.200$

  $P(credit\_rating = fair \mid buys\_computer = yes) = 6/9 = 0.667$

  $P(credit\_rating = fair \mid buys\_computer = no) = 2/5 = 0.400.$

Table 6.1 Class-labeled training tuples from the customer database of an electronics store.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- $P(\boldsymbol{X}|buys\_computer = yes) = P(age = youth \mid buys\_computer = yes)$
  $\times P(income = medium \mid buys\_computer = yes)$
  $\times P(student = yes \mid buys\_computer = yes)$
  $\times P(credit\_rating = fair \mid buys\_computer = yes)$
  $= 0.222 \times 0.444 \times 0.667 \times 0.667 = 0.044.$
- Similarly,
- $P(\boldsymbol{X}|buys\_computer = no) = 0.600 \times 0.400 \times 0.200 \times 0.400 = 0.019.$
- To find the class, C_i, that maximizes $P(\boldsymbol{X}|C_i)P(C_i)$, we compute
- $P(\boldsymbol{X}|buys\_computer = yes)P(buys\_computer = yes) = 0.044 \times 0.643 = 0.028$
- $P(\boldsymbol{X}|buys\_computer = no)P(buys\_computer = no) = 0.019 \times 0.357 = 0.007.$
- Therefore the naïve Bayesian classifier predicts *buys_computer = yes* for tuple $\boldsymbol{X}$.

Table 6.1 Class-labeled training tuples from the customer database of an electronics store.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- "*What if I encounter probability values of zero?*"

$$P(\boldsymbol{X}|Ci) = \prod_{k=1}^{n} P(x_k|C_i)$$
$$= P(x_1|C_i) \times P(x_2|C_i) \times \ldots \times P(x_n|C_i) \qquad (6.13)$$

  - What happens if we should end up with a probability value of zero for some $P(x_k|C_i)$? A zero probability cancels the effects of all the other (posteriori) probabilities (on C_i) involved in the product.
  - There is a simple trick to avoid this problem. We can assume that our training database, D, is so large that adding one to each count that we need would only make a negligible difference in the estimated probability value yet would conveniently avoid the case of probability values of zero.
  - This technique for probability estimation is known as the **Laplacian correction** or **Laplace estimator**, named after Pierre Laplace, a French mathematician who lived from 1749 to 1827.
  - If we have, say, q counts to which we each add one, then we must remember to add q to the corresponding denominator used in the probability calculation.

3. Bayes Classification Methods
3.2. Naïve Bayesian Classification

- **Example 6.6. Using the Laplacian correction to avoid computing probability values of zero.**
  - Suppose that for the class *buys_computer* = *yes* in some training database, *D*, containing 1000 tuples, we have 0 tuples with *income* = *low*, 990 tuples with *income* = *medium*, and 10 tuples with *income* = *high*.
  - The probabilities of these events, without the Laplacian correction, are 0, 0.990 (from 990/1000), and 0.010 (from 10/1000), respectively.
  - Using the Laplacian correction for the three quantities, we pretend that we have one more tuple for each income-value pair. In this way, we instead obtain the following probabilities (rounded up to three decimal places):

$$\frac{1}{1003} = 0.001, \quad \frac{991}{1003} = 0.988, \quad \text{and} \quad \frac{11}{1003} = 0.011,$$

respectively. The "corrected" probability estimates are close to their "uncorrected" counterparts, yet the zero probability value is avoided.

3. Bayes Classification Methods

3.2. Naïve Bayesian Classification

- The main idea of naïve Bayesian classifier lies in the class-conditional independence assumption, which significantly simplifies the estimation of the conditional probabilities $P(\boldsymbol{X}|C_i)$ ($i = 1, ..., m$).
- However, this (**class-conditional independence assumption**) is also one major limitation of naïve Bayesian classifier, since it might not be true for some applications.
- To address this issue, we need more sophisticated ways to approximate the conditional probabilities, such as Bayesian networks,

4. Lazy Learners (Learning From Your Neighbors)

- Eager learners, when given a set of training tuples, will construct a generalization (i.e., classification) model before receiving new (e.g., test) tuples to classify. We can think of the learned model as being ready and eager to classify previously unseen tuples.
  - Decision tree induction
  - Bayesian classification
- Imagine a contrasting lazy approach, in which the learner instead waits until the last minute before doing any model construction to classify a given test tuple. That is, when given a training tuple, a lazy learner simply stores it (or does only a little minor processing) and waits until it is given a test tuple.
  - Only when it sees the test tuple does it perform generalization to classify the tuple based on its similarity to the stored training tuples.
  - Unlike eager learning methods, lazy learners do less work when a training tuple is presented and more work when making a classification or numeric prediction.
  - Because lazy learners store the training tuples or "instances," they are also referred to as instance-based learners, even though all learning is essentially based on instances.

4. Lazy Learners (Learning From Your Neighbors)
4.1. *k*-Nearest-Neighbor Classifier

- Nearest-neighbor classifiers follow an idea of learning by analogy, that is, by comparing a given test tuple with training tuples that are similar to it.
- The training tuples are described by *n* attributes. Each tuple represents a point in an *n*-dimensional space. In this way, all the training tuples are stored in an *n*-dimensional attribute space.
- When given an unknown tuple, a ***k*-nearest-neighbor classifier** searches the attribute space for the *k* training tuples that are closest to the unknown tuple (i.e., to find your close friends). These *k* training tuples are the *k* "nearest neighbors" of the unknown tuple. Then *k-nearest-neighbor classifier* chooses the most common class label among the *k* nearest neighbors as the predicted class label of the unknown tuple (i.e., to follow the majority decision of your friends in the above example).
- "Closeness" is defined in terms of a distance metric, such as Euclidean distance.

4. Lazy Learners (Learning From Your Neighbors)
4.1. *k*-Nearest-Neighbor Classifier

- The Euclidean distance between two points or tuples $\boldsymbol{X}_1 = (x_{11}, x_{12}, \ldots, x_{1n})$ and $\boldsymbol{X}_2 = (x_{21}, x_{22}, \ldots, x_{2n})$, is

$$dist(\boldsymbol{X}_1, \boldsymbol{X}_2) = \sqrt{\sum_{i=1}^{n} (x_{1i} - x_{2i})^2} \quad (6.17)$$

  - Typically, we normalize the values of each attribute before using Eq. (6.17).

- For *k*-nearest-neighbor classification, the unknown tuple is assigned the most common class label among its *k*-nearest neighbors.
  - When $k = 1$, the unknown tuple is assigned the class of the training tuple that is closest to it in the attribute space.
  - When $k > 1$, we can take a (weighted) majority voting on the class labels among its *k*-nearest neighbors.

- Nearest-neighbor classifiers can also be used for numeric prediction, that is, to return a real-valued prediction for a given unknown tuple. In this case, the classifier returns the (weighted) average value of the real-valued labels associated with the *k*-nearest neighbors of the unknown tuple.

4. Lazy Learners (Learning From Your Neighbors)

4.1. *k*-Nearest-Neighbor Classifier

- "*But how can distance be computed for attributes that are not numeric, but nominal (or categorical) such as color?*"
- For nominal attributes, a simple method is to compare the corresponding value of the attribute in tuple $\boldsymbol{X}_1$ with that in tuple $\boldsymbol{X}_2$.
  - If the two are identical, then the difference between the two is taken as 0. If the two are different, then the difference is considered to be 1.
  - Other methods may incorporate more sophisticated schemes for differential grading (e.g., where a larger difference score is assigned, say, for blue and white than for blue and black).

4. Lazy Learners (Learning From Your Neighbors)
4.1. *k*-Nearest-Neighbor Classifier

- "*What about missing values?*"
- In general, if the value of a given attribute A is missing in tuple $\boldsymbol{X}_1$ or in tuple $\boldsymbol{X}_2$, **we assume the maximum possible difference**.
- Suppose that each of the attributes has been mapped to the range [0, 1].
- For nominal attributes, we take the difference value to be 1 if either one or both of the corresponding values of A are missing.
- If A is numeric and missing from both tuples $\boldsymbol{X}_1$ and $\boldsymbol{X}_2$, then the difference is also taken to be 1.
- If only one value is missing and the other (which we will call v') is present and normalized, then we can take the difference to be either $|1 - v'|$ or $|0 - v'|$ (i.e., $1 - v'$ or v), whichever is greater.

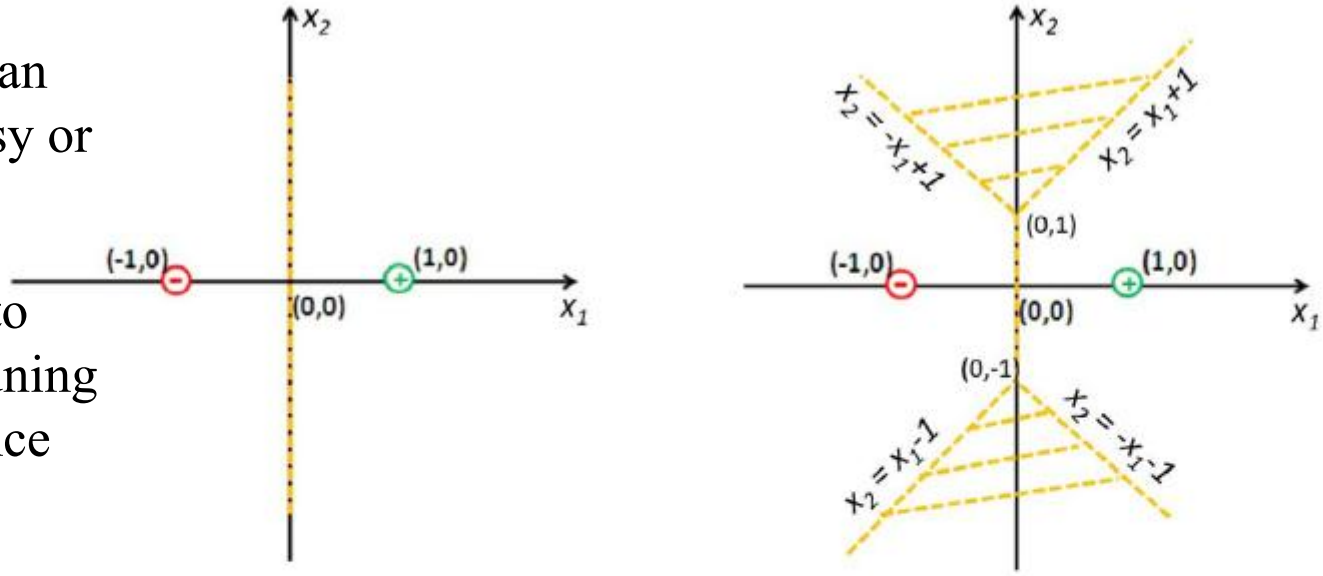
4. Lazy Learners (Learning From Your Neighbors)

4.1. *k*-Nearest-Neighbor Classifier

- "*How can I determine a good value for k, the number of neighbors?*"
- This can be determined experimentally.
  - Starting with $k = 1$, we use a test set to estimate the error rate of the classifier.
  - This process can be repeated each time by incrementing k to allow for one more neighbor.
  - The k value that gives the minimum error rate may be selected.
  - In general, the larger the number of training tuples, the larger the value of k will be (so that classification and numeric prediction decisions can be based on a larger portion of the stored tuples).
  - As the number of training tuples approaches infinity and $k = 1$, the error rate can be no worse than twice the Bayes error rate (the latter being the theoretical minimum). In other words, 1-nearest-neighbor classifier is asymptotically near-optimal. If k approaches infinity, the error rate approaches the Bayes error rate.

4. Lazy Learners (Learning From Your Neighbors)
4.1. *k*-Nearest-Neighbor Classifier

- Nearest-neighbor classifiers use distance-based comparisons that intrinsically assign equal weight to each attribute. They, therefore, can suffer from poor accuracy when given noisy or irrelevant attributes.
- The method, however, has been modified to incorporate attribute weighting and the pruning of noisy data tuples. The choice of a distance metric can be critical. The Manhattan (city block) distance, or other distance measurements, may also be used.



(a) decision boundary using L_2 norm

(b) decision boundary using L_∞ norm.

FIGURE 6.9

The impact of distance metrics on 1-nearest-neighbor classifier. Given two training examples, including a positive example at $(1, 0)$ and a negative example at $(-1, 0)$. The decision boundaries of 1-nearest-neighbor classifier using different distance metrics are quite different from each other. Using L_2 norm (on the left), the decision boundary is a vertical line at $x_2 = 0$. Using L_∞ norm (on the right), the decision boundary includes a line segment between $(0, -1)$ and $(0, 1)$ and two shaded areas.

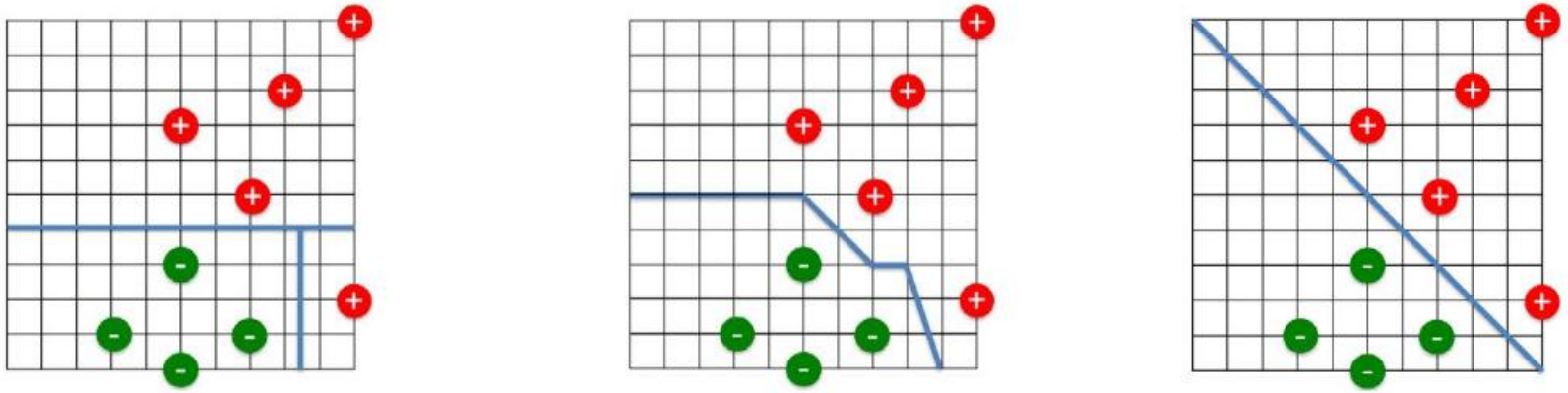
4. Lazy Learners (Learning From Your Neighbors)

4.1. *k*-Nearest-Neighbor Classifier

- Nearest-neighbor classifiers can be extremely slow when classifying test tuples.
- If D is a training database of $|D|$ tuples and $k = 1$, then $O(|D|)$ comparisons are required to classify a given test tuple.
- By presorting and arranging the stored tuples into search trees, the number of comparisons can be reduced to $O(log(|D|)$.
- Parallel implementation can reduce the running time to a constant, that is, $O(1)$, which is independent of $|D|$.

5. Linear Classifiers

- So far, we have learned a few classifiers which are capable of generating complex decision boundaries.
- For example, a decision tree classifier might output a hyperrectangular-shaped decision boundary (Fig. 6.10(a)), and a *k*-nearest-neighbor classifier might output a hyperpolygonal-shaped decision boundary (Fig. 6.10(b)). However, what about a simple, linear decision boundary? .



(a) Decision Tree Classifier (b) 1−Nearest-Neighbor Classifier (c) Linear Classifier

FIGURE 6.10

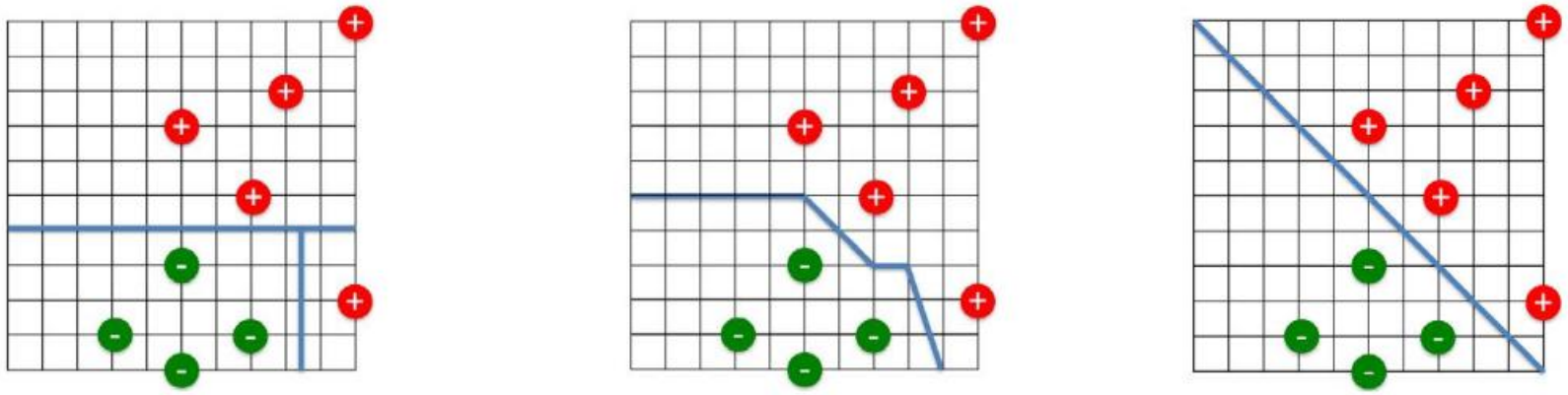
Decision boundaries by different classifiers. Note that this example is *linearly separable*, meaning that a linear classifier (c) can perfectly separate all the positive training tuples from all the negative training tuples. If the training set is *linearly inseparable*, we could still use a linear classifier, at the expense that some training tuples are on the 'wrong' side of the decision boundary. In Chapter 7, we will introduce techniques (e.g., support vector machines) to handle linearly inseparable case.

5. Linear Classifiers

- Linear regression
- Perceptron
- Logistic regression

5. Linear Classifiers

- For the example in Fig. 6.10, intuitively, a linear decision boundary (the straight line in Fig. 6.10(c)) is (almost) as good as decision tree classifiers and *k*-nearest-neighbor classifier in separating the positive training tuples from the negative training ones.
  - Yet, such a linear decision boundary might offer additional advantages, such as efficient computation for training the classifier, better generalization performance, and better interpretability.



(a) Decision Tree Classifier (b) 1−Nearest-Neighbor Classifier (c) Linear Classifier

FIGURE 6.10

Decision boundaries by different classifiers. Note that this example is *linearly separable*, meaning that a linear classifier (c) can perfectly separate all the positive training tuples from all the negative training tuples. If the training set is *linearly inseparable*, we could still use a linear classifier, at the expense that some training tuples are on the 'wrong' side of the decision boundary. In Chapter 7, we will introduce techniques (e.g., support vector machines) to handle linearly inseparable case.

5. Linear Classifiers
5.1. Linear Regression

- Linear regression is a statistical technique that predicts a **continuous value** based on one or more independent attributes.
  - For example, we might want to predict the housing price based on the living area or to predict the future income of a student based on which college she attended, in which major and the overall GPA, etc.
- Since linear regression aims to predict a continuous value, it cannot be directly applied to the classification task, where the output is a categorical variable.
  - Nonetheless, the core techniques in linear regression form the basis of linear classifiers.

5. Linear Classifiers
5.1. Linear Regression

- Suppose we have n tuples, each of which is represented by p attributes $x_i = (x_{i,1}, ..., x_{i,p})^T$ and a continuous output value y_i ($i = 1, ..., n$).
- In linear regression, we want to learn a linear function that maps the p input attributes x_i to the output variable y_i, that is,

$$\hat{y}_i = w^T x_i + b = \sum_{j=1}^{p} w_j x_{i,j} + b$$

$\hat{y}_i$ is the predicted output value for the ith tuple, $w = (w_1, ..., w_p)^T$ is a p-dimensional weight vector and b is the bias scalar.

  - In other words, linear regression assumes that the output value is a linear weighted summation of the p input attribute values, offset by the bias scalar b.
  - The entries in the weight vector w_j ($j = 1, ..., p$) tell how important the corresponding attribute $x_{i,j}$ is in predicting the output variable $\hat{y}_i$.

5. Linear Classifiers
5.1. Linear Regression

- "*So, how can we determine the weight vector w and the bias scalar b?*"
- Intuitively, we want to learn the "best" weight vector *w* and the "best" bias scalar *b* from the training data, so that the linear regression model can make the "best" prediction.
  - That is, the predicted value $\hat{y} = w^T x_i + b$ is as close as possible to the actual observed value y_i (i = 1, ..., n).
- One of the most common linear regression methods is called **least square regression**, which aims to minimize the following loss function $L(w, b) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} (y_i - (w^T x_i + b))^2$.

5. Linear Classifiers
5.1. Linear Regression

- $p = 1$: There is only one input attribute.
  - The optimal weight vector w:

$$w = \frac{\sum_{i=1}^{n} x_i(y_i - \bar{y})}{\sum_{i=1}^{n} x_i^2 - \frac{1}{n}(\sum_{i=1}^{n} x_i)^2}$$

  - The optimal bias scalar b:

$$b = \frac{1}{n}\sum_{i=1}^{n}(y_i - wx_i)$$

$$\text{where } \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

5. Linear Classifiers
5.1. Linear Regression

- $p > 1$: There are multiple p attributes.
  - We assume there is an additional "dummy" attribute which always takes the value of 1 for any tuple.
    - Input attribute $(p + 1)$-dimensional vector: $x_i = (1, x_{i,1}, ..., x_{i,p})$
  - Let the weight for this dummy attribute be w_0.
    - Then the overall weight vector $w = (w_0, w_1, ..., w_p)$
  - The multilinear regression model can be re-written as

$$\hat{y}_i = w^T x_i = \sum_{j=0}^{p} w_j x_{i,j} + b = w_0 + w_1 x_{i,1} + w_2 x_{i,2} + \ldots + w_p x_{i,p.}$$

  - We use the same lost function

$$L(w) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{n} (y_i - (w^T x_i + b))^2$$

5. Linear Classifiers
5.1. Linear Regression

- The optimal weight vector w:

$$w = (XX^T)^{-1}Xy$$

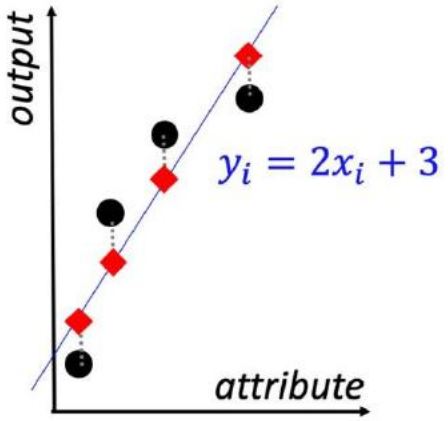
- where
  - $X = [x_1, x_2, \ldots, x_n]$ $(p+1) \times n$ matrix,
  - $y = [y_1, y_2, \ldots, y_n]^T$ n × 1 vector.

5. Linear Classifiers
5.1. Linear Regression

- **Example 6.7.** Let us look at an example of least square regression in Fig. 6.11.
- There are four training tuples, each represented by a single-dimensional attribute x_i and an output variable y_i ($i = 1, 2, 3, 4$).
- We want to find least square regression model $y = wx + b$ that predicts the output y based on the input attribute x.

Index (i)	1	2	3	4
Attribute (x_i)	1	3	5	7
Output (y_i)	4	10	14	16

(a) Training tuples



(b) Least square regression

FIGURE 6.11

An example of least square regression. (a) Four training tuples. (b) Scatter-plot of the training tuples (black dots) and least square regression model (the blue line). Red diamonds are the predicted output $\hat{y}_i$ ($i = 1, 2, 3, 4$) and dashed lines indicate the prediction errors ($|y_i - \hat{y}_i|$) of the corresponding training tuples.

5. Linear Classifiers
5.2. Perceptron: Turning Linear Regression to Classification

- "*How can we modify a linear regression model to perform classification task?*"
- Suppose we have a binary classification task. The output value y_i for a given tuple is a binary variable:
  - $y_i = 1$ indicates the ith tuple is a positive tuple and
  - $y_i = 0$ indicates the ith tuple is a negative one.
- One way to modify the linear regression model for such a binary classification task is to use the *sign* of the output of the linear regression model as the predicted class label, that is,

$$\hat{y}_i = \text{sign}(w^T x_i)$$

  where $\hat{y}_i$ is the predicted class label for ith tuple, sign(z) = 1 if $z > 0$ and sign(z) = 0 otherwise.
- Notice that we use the same notation as multilinear regression where we have introduced a "dummy" attribute which always takes the value of 1 for any tuple.
- Therefore if we know the weight vector w, we can use it to predict the class label of a given tuple as follows. We compute a linear combination of the attribute values of the given tuple, weighted by the corresponding entries of the weight vector w. If the resulting value of such a linear combination is positive, we predict that the given tuple is a positive tuple. Otherwise, we predict that it is a negative one.

5. Linear Classifiers
5.2. Perceptron: Turning Linear Regression to Classification

- "*How can we find the optimal weight vector w from a set of training tuples?*"
- The classic learning algorithm to train a perceptron is as follows.
  - We start with an initial guess of the weight vector w (e.g.,we can simply set $w = 0$). Then, the learning algorithm will iterate until it converges, or the maximum iteration number or some other preset stopping criteria are met.
  - In each iteration, we do the following for each training tuple x_i.
    - We try to predict the class label of x_i using the current weight vector w, that is, $\hat{y}_i = \text{sign}(w^T x_i)$.
    - If the prediction is correct (i.e., $\hat{y}_i = y_i$), we do nothing about the weight vector.
    - However, if the prediction is incorrect (i.e., $\hat{y}_i \neq y_i$), we update the current weight vector in one of the following two ways.
      - If $y_i = +1$ (i.e., the ith tuple is a positive tuple, but the current classifier predicts it is a negative tuple), we update weight vector as $w \leftarrow w + \eta x_i$.
      - If $y_i = 0$ (i.e., the ith tuple is a negative tuple, which is wrongly predicted by the current classifier as a positive tuple), we update weight vector as $w \leftarrow w - \eta x_i$, where $\eta > 0$ is the user-specified learning rate.

5. Linear Classifiers

5.2. Perceptron: Turning Linear Regression to Classification

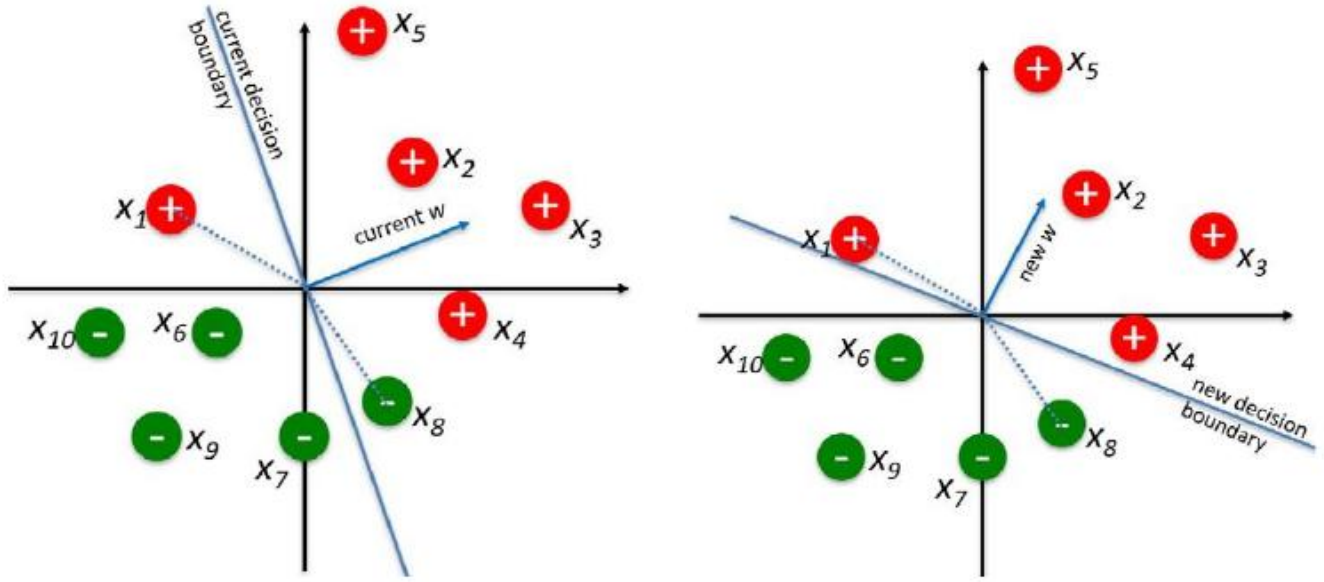
- So, the intuition is that in each iteration of the training process, the algorithm will focus on those wrongly predicted training tuples by the current weight vector w.
- If the wrongly predicted training tuple x_i is a positive tuple, we update the weight vector w by moving it towards the attribute vector x_i of this training tuple (i.e., $w \leftarrow w + \eta x_i$).
- On the other hand, if the wrongly predicted training tuple x_i is a negative tuple, we update the weight vector w by moving it away from the attribute vector x_i of this training tuple (i.e., $w \leftarrow w - \eta x_i$).

5. Linear Classifiers
5.2. Perceptron: Turning Linear Regression to Classification

- **Example 6.8.** Let us look at an example in Fig. 6.12 for training a perceptron classifier.
- In Fig. 6.12, we assume the bias $w_0 = 0$ for illustration clarity.
- Fig. 6.12(a) (left) shows the current decision boundary and the weight vector w, where two training tuples are wrongly classified, including a positive tuple x_1 and a negative tuple x_8.
- Therefore only these two tuples are used to update the weight vector in the current iteration, that is,

$$w \leftarrow w + \eta x_1 - \eta x_8.$$

- The updated weight vector w and the corresponding decision boundary are shown in Fig. 6.12(b) (right), where all training tuples are correctly classified.



(a) current weight vector w (b) updated weight vector w

FIGURE 6.12

5. Linear Classifiers

5.2. Perceptron: Turning Linear Regression to Classification

- "*How effective is the perceptron learning algorithm?*"
- If the training tuples are linearly separable (e.g., the example in Fig. 6.12), the perceptron algorithm is guaranteed to find a weight vector (i.e., a hyperplane decision boundary) that perfectly separates all the positive training tuples from all the negative training tuples.
- However, if the training tuples are not linearly separable, this algorithm will fail to converge.
- Perceptron, one of the earliest linear classifiers, was first invented back in 1958. It can also be used as a building block (called a "neuron") in deep neural networks.

5. Linear Classifiers
5.3. Logistic Regression

- Perceptron that we have just introduced in the previous section is capable of predicting the binary class label of a given tuple. **However, can we also tell how confident such a prediction is?**
- Again, let us consider a binary classification task, and we assume that there are two possible class labels, that is, $y = 1$ for a positive tuple and y = 0 for a negative tuple.
- Recall that in (naïve) Bayes classifier, we can estimate the posterior probability $P(y_i = 1|x_i)$, which can be directly used to indicate how confident the predicted classification result is.
  - For example, if $P(y_i = 1|x_i)$ is close to 1, the classifier is highly confident that the tuple x_i is a positive example.
- *How can we make a linear classifier not only predict which class label a tuple has, but also tell how confident it is in making such a prediction?*
  - An effective way to this end is via **logistic regression** classifier. Let us first introduce an important function called **sigmoid** function, which is defined as

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

5. Linear Classifiers
5.3. Logistic Regression

- **Sigmoid** function, which is defined as

$$\sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^{z}}{1+e^{z}}$$

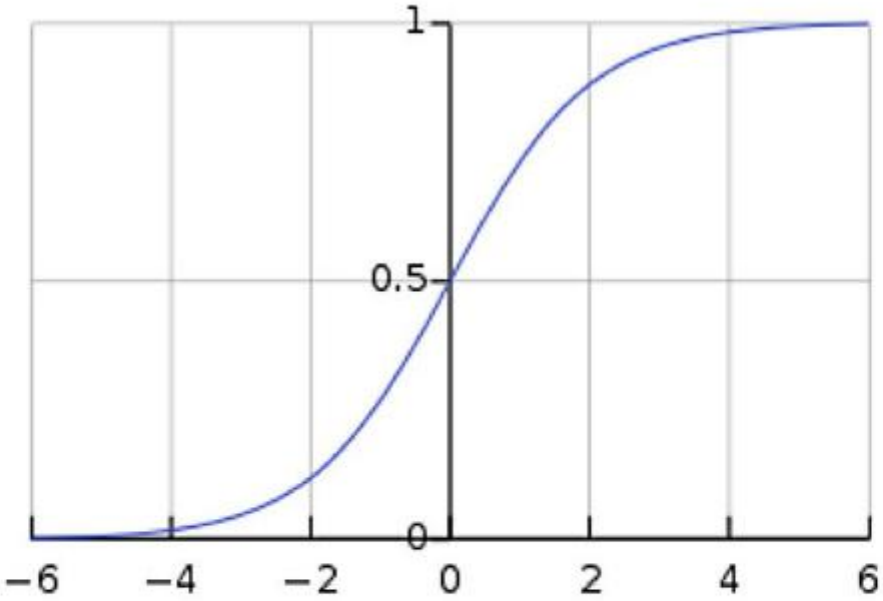


FIGURE 6.13

Illustration of sigmoid function. The sigmoid function "squashes" an input from a larger range $(-\infty, +\infty)$ to a smaller range (0, 1). For this reason, sigmoid function is also called *squash function*. In Chapter 10, we will see other types of squash functions, which are called activation functions in the deep learning terminology.

5. Linear Classifiers
5.3. Logistic Regression

- The sigmoid function maps a real number in $(-\infty, +\infty)$ to an output value in the range of (0, 1).
- Therefore if we leverage the sigmoid function to map the output of a linear regression model to a number between 0 and 1, we can interpret the mapping result as the posterior probability of observing a positive class label.
- This is exactly what logistic regression classifier tries to do!
- Formally, we have

$$P(\hat{y}_i = 1 | x_i, w) = \sigma(w^T x_i) = \frac{1}{1 + e^{-w^T x_i}}$$

  - If $P(\hat{y}_i = 1 | x_i, w) > 0.5$, the classifier predicts that the tuple x_i is a positive tuple (i.e., $\hat{y}_i = 1$), otherwise, it predicts a negative tuple (i.e., $\hat{y}_i = 0$).

5. Linear Classifiers
5.3. Logistic Regression

- "*How can we determine the optimal weight vector w from a set of training tuples?*"
- The classic method to train a logistic regression classifier is via **maximum likelihood estimation** (MLE).
- Let us assume there are n training tuples (x_i, y_i) $(i = 1, ..., n)$.
- Since we have a binary classification task, we can view the predicted class label $\hat{y}_i$ as a Bernoulli random variable, which can only take two possible values, including

$$P(\hat{y}_i = 1|x_i, w) = p_i \text{ and } P(\hat{y}_i = 0|x_i, w) = 1 - p_i, \text{ where } p_i = \sigma(w^T x_i) = \frac{1}{1+e^{-w^T x_i}}$$

- Notice that the true class label y_i for the ith tuple is a binary variable. Therefore we have that

$$P(\hat{y}_i = y_i) = p_i^{y_i}(1 - p_i)^{1-y_i}$$

5. Linear Classifiers
5.3. Logistic Regression

- The maximum likelihood estimation method aims to solve the following optimization problem, which says that we should choose the best weight vector w that maximizes the likelihood of the training set.
- The intuition is that we want to find the optimal model parameter (i.e., the weight vector w) so that there is the highest "chance" (i.e., the likelihood or the probability) of observing the entire training set.

$$w^* = \text{argmax}_w \, L(w) = \Pi_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1-y_i} = \Pi_{i=1}^{n} \left(\frac{e^{w^T x_i}}{1 + e^{w^T x_i}}\right)^{y_i} \left(\frac{1}{1 + e^{w^T x_i}}\right)^{1-y_i} \quad (6.19)$$

5. Linear Classifiers
5.3. Logistic Regression

- "*But, how can we develop an algorithm to solve this optimization problem to find the optimal weight vector w?*"
- First, we notice that the likelihood function $L(w)$ has many nonnegative terms that are multiplied with each other.
- In practice, it is often more convenient to work with the logarithm of such a complicated function. Thus we have the following equivalent optimization problem, where $l(w)$ is called the log likelihood

$$w^* = \text{argmax}_w \, l(w) = \sum_{i=1}^{n} y_i x_i^T w - \log(1 + e^{w^T x_i}). \qquad (6.20)$$

5. Linear Classifiers
5.3. Logistic Regression

- A common strategy is to find the optimal solution w^* iteratively as follows.
  - In each iteration, we try to improve the current weight vector w so that the objective function we wish to maximize (the log likelihood function $l(w)$) is improved most.
  - In order to increase the current objective function $l(w)$ most, it turns out the best direction to update the current estimation of the weight vector w is to follow its gradient. This leads to the following algorithm to learn the optimal weight vector w^* from the training set.
  - We start with an initial guess of the weight vector w (e.g., we can simply set $w = 0$). Then, the learning algorithm will iterate until it converges, or the maximum iteration number or some other preset stopping criteria are met.
  - In each iteration, it updates the weight vector w as follows

$$w \leftarrow w + \eta \sum_{i=1}^{n} (y_i - P(\hat{y} = 1|x_i, w))x_i$$

where $\eta > 0$ is the user-specified learning rate.

5. Linear Classifiers
5.3. Logistic Regression

- "*So, what is the intuition of the above algorithm?*"
- Let us analyze the impact of each training tuple (x_i, y_i) on updating the estimation of the weight vector w. We consider two situations depending on whether it is a positive tuple (i.e., $y_i = 1$) or a negative tuple (i.e., $y_i = 0$).
  - $y_i = 1$: $w \leftarrow w + \eta(1 - P(\hat{y} = 1|x_i, w))x_i$
    - The intuition is that we want update the current weight vector w *towards* the direction of the attribute vector x_i of this positive tuple.
  - $y_i = 0$: $w \leftarrow w - \eta P(\hat{y} = 1|x_i, w)x_i$
    - The intuition is that we want update the current weight vector w *away from* the direction of the attribute vector x_i of this positive tuple.

5. Linear Classifiers
5.3. Logistic Regression

- "*How good is the logistic regression algorithm? What are the potential limitations and how to mitigate?*"

- 1. The learned weight vector becomes too large.
  - Since the log likelihood function $l(w)$ is a concave function, the algorithm for training a logistic regression classifier described above is guaranteed to converge to its optimal solution.
  - However, if the training set is linearly separable, the algorithm might converge to a weight vector w with an infinitely large norm.
    - A "large" weight vector w could make the trained classifier prone to the noise of certain attributes of a given tuple. This will, in turn, lead to a poor generalization performance of the learned logistic regression classifier.
    - In other words, the learned logistic regression classifier overfits the training set.

5. Linear Classifiers
5.3. Logistic Regression

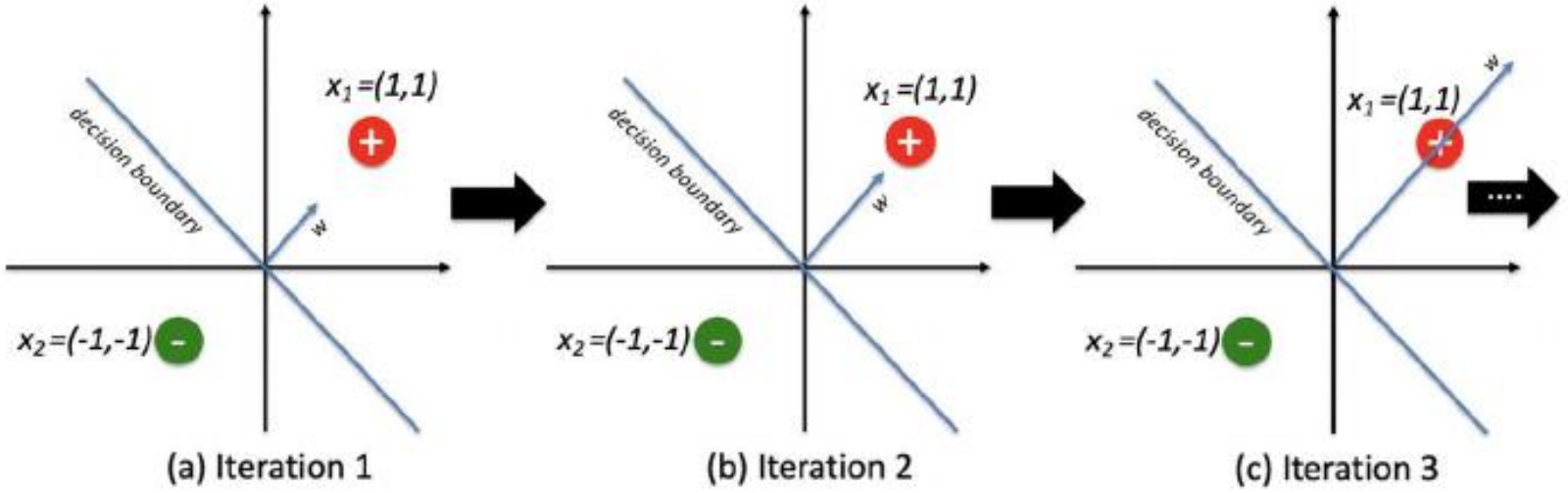


FIGURE 6.14

Illustration of the infinitely large weight vector of logistic regression in linearly separable case. There are two training tuples in 2d space, with one positive training tuple $x_1 = (1, 1)$ and one negative training tuple $x_2 = (-1, -1)$. For simplicity, we let the bias scalar $b = 0$ and the learning rate $\eta = 1$. Suppose that at iteration 1, the weight vector $w = (1, 1)$. Then, logistic regression algorithm introduced above will update the weight vector as $w_{\text{new}} = w_{\text{old}} + (1 - P(\hat{y}_1 = 1|x_1, w_{\text{old}}))x_1 - P(\hat{y}_2 = 1|x_2, w_{\text{old}})x_2 = (0.5, 0.5) + a(1, 1) = (1 + 2a)w_{\text{old}}$, where $a = 1 - P(\hat{y}_1 = 1|x_1, w_{\text{old}}) + P(\hat{y}_2 = 1|x_2, w_{\text{old}}) > 0$. As such, the new weight vector w_{new} shares the same direction as the old w_{old}. Therefore, the decision boundary remains the same, but new weight vector w_{new} grows in the magnitude by a factor of $(1 + 2a)$. This trend will continue as the logistic regression algorithm progresses, leading to a weight vector with an infinitely large magnitude.

5. Linear Classifiers
5.3. Logistic Regression

- 2. The second potential limitation is the *independence* assumption behind logistic regression. Recall that when we calculate the likelihood $L(w)$ of the training set, we simply multiply the likelihood of each training tuple together. This means that we have implicitly assumed that different training tuples are independent of each other.

$$w^* = \text{argmax}_w \, L(w) = \Pi_{i=1}^{n} p_i^{y_i} (1 - p_i)^{1-y_i} = \Pi_{i=1}^{n} \left(\frac{e^{w^T x_i}}{1 + e^{w^T x_i}}\right)^{y_i} \left(\frac{1}{1 + e^{w^T x_i}}\right)^{1-y_i} \qquad (6.19)$$

  - However, this assumption might be violated in some applications (e.g., users on a social network are interconnected with each other). The graph-based classification might provide a natural remedy for this issue.

5. Linear Classifiers
5.3. Logistic Regression

- 3. The third potential limitation lies in the computational challenge. Notice that in the updating rule

$$w \leftarrow w + \eta \sum_{i=1}^{n} (y_i - P(\hat{y} = 1|x_i, w))x_i$$

described above, we need to calculate the gradients $(y_i - P(\hat{y} = 1|x_i, w))$ for all training tuples and then sum them up to update the weight vector *w*.

  - If there are millions of training tuples, it is computationally very expensive to perform such computation. An efficient way to address this issue is to use *stochastic gradient descent* method to train a logistic regression classifier.
  - That is, at each iteration, we will randomly sample a small subset of training tuples (this is often referred to as a *minibatch*) and *only* use the sampled tuples (instead of all training tuples) to update the weight vector.

6. Model Evaluation and Selection
6.1. Metrics for Evaluating Classifier Performance

- *How good or how "accurate" is a classifier at predicting the class label of tuples?*
- We will consider the case where the class tuples are more or less evenly distributed, as well as the case where classes are unbalanced (e.g., where an important class of interest is rare such as in medical tests).
- Using training data to derive a classifier and then estimate the accuracy of the learned model can result in misleading overoptimistic estimates due to overspecialization of the learning algorithm to the data.
  - Instead, it is better to measure the classifier's accuracy on a test set consisting of class-labeled tuples that were not used to train the model.

6. Model Evaluation and Selection
6.1. Metrics for Evaluating Classifier Performance

- **Positive tuples**: Tuples of the main class of interest.
  - P: The number of positive tuples
- **Negative tuples**: All other tuples.
  - N: The numberof negative tuples
- **True positives** (TP): These refer to the positive tuples that were correctly labeled by the classifier.
  - TP: The number of true positives
- **True negatives** (TN): These are the negative tuples that were correctly labeled by the classifier.
  - TN: The number of true negatives
- **False positives** (FP): These are the negative tuples that were incorrectly labeled as positive.
  - FP: The number of false positives
- **False negatives** (FN): These are the positive tuples that were mislabeled as negative.
  - FN: The number of false negatives

FIGURE 6.16

Confusion matrix, shown with totals for positive and negative tuples.

		Predicted class		
		yes	*no*	Total
Actual class	*yes*	TP	FN	P
	no	FP	TN	N
	Total	P'	N'	$P + N$

6. Model Evaluation and Selection
6.1. Metrics for Evaluating Classifier Performance

- A confusion matrix is a useful tool for analyzing how well your classifier can recognize tuples of different classes.
  - TP and TN tell us when the classifier is getting things right, whereas FP and FN tell us when the classifier is getting things wrong (i.e., mislabeling).

		Predicted class		
		yes	*no*	Total
Actual class	*yes*	TP	FN	P
	no	FP	TN	N
	Total	P'	N'	$P + N$

- Given m classes (where $m \geq 2$), a **confusion matrix** CM is a table of at least size m by m. An entry, $CM_{i,j}$ at the ith row and the jth column indicates the number of tuples of class i that were labeled by the classifier as class j.
- For a classifier to have good accuracy, ideally most of the tuples would be represented along the diagonal of the confusion matrix, from entry $CM_{i,i}$ to entry $CM_{m,m}$, with the rest of the entries being zero or close to zero. That is, ideally, FP and FN are around zero.

6. Model Evaluation and Selection
6.1. Metrics for Evaluating Classifier Performance

- Evaluation measures

Measure	Formula
accuracy, recognition rate	$\frac{TP+TN}{P+N}$
error rate, misclassification rate	$\frac{FP+FN}{P+N}$
sensitivity, true positive rate, recall	$\frac{TP}{P}$
specificity, true negative rate	$\frac{TN}{N}$
precision	$\frac{TP}{TP+FP}$
F, F_1, F-score, harmonic mean of precision and recall	$\frac{2 \times precision \times recall}{precision + recall}$
F_β, where β is a nonnegative real number	$\frac{(1+\beta^2) \times precision \times recall}{\beta^2 \times precision + recall}$

FIGURE 6.15

Evaluation measures. Note that some measures are known by more than one name. TP, TN, FP, FN, P, N refer to the number of true positive, true negative, false positive, false negative, positive, and negative samples, respectively (see text).

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Pe

Accuracy (Recognition Rate)

		Predicted class		
		yes	*no*	Total
Actual class	*yes*	TP	FN	P
	no	FP	TN	N
	Total	P'	N'	$P + N$

- The accuracy of a classifier on a given test set is the percentage of test set tuples that are correctly classified by the classifier.

$$accuracy = \frac{TP + TN}{P + N}$$

  - Accuracy is most effective when the class distribution is relatively balanced.

Classes	*buys_computer* = *yes*	*buys_computer* = *no*	Total	Recognition (%)
buys_computer = *yes*	**6954**	**46**	7000	99.34
buys_computer = *no*	**412**	**2588**	3000	86.27
Total	7366	2634	10,000	95.42

FIGURE 6.17

Confusion matrix for the classes *buys_computer* = *yes* and *buys_computer* = *no*, where an entry in row i and column j shows the number of tuples of class i that were labeled by the classifier as class j. Ideally, the nondiagonal entries should be zero or close to zero.

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Pe

Error Rate (Misclassification Rate)

		Predicted class		
		yes	*no*	Total
Actual class	*yes*	TP	FN	P
	no	FP	TN	N
	Total	P'	N'	$P + N$

- The error rate or misclassification rate of a classifier, ***M***, which is simply $1 - accuracy(\boldsymbol{M})$, where $accuracy(\boldsymbol{M})$ is the accuracy of ***M***. This also can be computed as

$$error\ rate = \frac{FP + FN}{P + N}$$

- If we were to use the training set (instead of a test set) to estimate the error rate of a model, this quantity is known as the **resubstitution error**.
  - This error estimate is optimistic of the true error rate (and similarly, the corresponding accuracy estimate is optimistic) because the model is not tested on any samples that it has not already seen.

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Performance

Error Rate (Misclassification Rate)

- Class imbalance problem
  - The main class of interest is rare. The data set distribution reflects a significant majority of the negative class and a minority positive class.
  - For example, in fraud detection applications, the class of interest (or positive class) is "*fraud*" which occurs much less frequently than the negative "*nonfraudulant*" class. In medical data, there may be a rare class, such as "*cancer*."
  - Suppose that you have trained a classifier to classify medical data tuples, where the class label attribute is "*cancer*" and the possible class values are "*yes*" and "*no*." An accuracy rate of, say, 97% may make the classifier seem quite accurate, but what if only, say, 3% of the training tuples are actually cancer?
    - Clearly, an accuracy rate of 97% may not be acceptable—the classifier could be correctly labeling only the noncancer tuples, for instance, and misclassifying all the cancer tuples.
    - Instead, we need other measures, which assess how well the classifier can recognize the positive tuples (*cancer* = *yes*) and how well it can recognize the negative tuples (*cancer* = *no*).
      - The **sensitivity** and **specificity** measures can be used, respectively, for this purpose.

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Pe

Sensitivity and Specificity

		Predicted class		
		yes	*no*	Total
Actual class	*yes*	TP	FN	P
	no	FP	TN	N
	Total	P'	N'	$P + N$

- Sensitivity is also referred to as the *true positive* (*recognition*) *rate.*
  - The proportion of positive tuples that are correctly identified.

$$sensitivity = \frac{TP}{P}$$

- Specificity is the *true negative rate*
  - The proportion of negative tuples that are correctly identified.

$$specificity = \frac{TN}{N}$$

- It can be shown that accuracy is a function of sensitivity and specificity:

$$accuracy = sensitivity\frac{P}{P+N} + specificity\frac{N}{P+N}$$

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Performance

Sensitivity and Specificity

- **Example 6.9. Sensitivity and specificity.**

Classes	*yes*	*no*	Total	Recognition (%)	
yes	90	210	300	30.00%	Sensitivity
no	140	9560	9700	98.56%	Specificity
Total	230	9770	10000	96.50%	

  - Thus we note that although the classifier has a high accuracy, it's ability to correctly label the positive (rare) class is poor given its low sensitivity. It has high specificity, meaning that it can accurately recognize negative tuples.

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Pe

Precision and Recall

		Predicted class		
		yes	*no*	Total
Actual class	*yes*	TP	FN	P
	no	FP	TN	N
	Total	P'	N'	$P + N$

- **Precision** can be thought of as a measure of *exactness* (i.e., what percentage of tuples labeled as positive are actually such).

$$precision = \frac{TP}{TP + FP}$$

- **Recall** is a measure of *completeness* (what percentage of positive tuples are labeled as such).

$$recall = \frac{TP}{TP + FN} = \frac{TP}{P} = sensitivity$$

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Performance

Precision and Recall

- **Example 6.10. Precision and recall.**

Classes	*yes*	*no*	Total	Recognition (%)	
yes	90	210	300	**30.00%**	**Recall/Sensitivity**
no	140	9560	9700	98.56%	Specificity
	39.13%		10000	96.50%	Accuracy
	Precision				

- A perfect precision score of 1.0 for a class *C* means that every tuple that the classifier labeled as belonging to class *C* does indeed belong to class *C*. However, it does not tell us anything about the number of class *C* tuples that the classifier mislabeled.
- A perfect recall score of 1.0 for *C* means that every item from class *C* was labeled as such, but it does not tell us how many other tuples were incorrectly labeled as belonging to class *C*.
- There tends to be an inverse relationship between precision and recall, where it is possible to increase one at the cost of reducing the other.
  - For example, our medical classifier may achieve high precision by labeling all *cancer* tuples that present a certain way as *cancer* but may have low recall if it mislabels many other instances of *cancer* tuples.
- Precision and recall scores are typically used together, where precision values are compared for a fixed value of recall, or vice versa. For example, we may compare precision values at a recall value of, say, 0.75.

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Performance

F_1 and F_β

- An alternative way to use precision and recall is to combine them into a single measure. This is the approach of the F measure (also known as the F_1 score or F-score) and the F_β measure.

$$F = \frac{2 \times precision \times recall}{precision + recall}$$

$$F_\beta = \frac{(1+\beta^2) \times precision \times recall}{\beta^2 \times precision + recall}$$ β is a nonnegative real number.

- The F measure is the *harmonic mean* of precision and recall. It gives equal weights to precision and recall.

- The F_β measure is a weighted measure of precision and recall. It assigns β^2 times as much weight to recall as to precision. Commonly used F_β measures are F_2 (which weights recall twice as much as precision) and $F_{0.5}$ (which weights precision twice as much as recall).

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Performance

F_1 and F_β

- **Harmonic mean** of a data set $x_1, x_2, \ldots, x_n$, is the reciprocal of the arithmetic mean of the reciprocals of the numbers $x_1, x_2, \ldots, x_n$,.

$$H(x_1, x_2, \ldots, x_n) = \left(\frac{\sum_{i=1}^{n} \frac{1}{x_i}}{n} \right)^{-1} = \frac{n}{\sum_{i=1}^{n} \frac{1}{x_i}}$$

- **Weighted harmonic mean**: If a set of weights $w_1, w_2, \ldots, w_n$ is associated to the data set $x_1, x_2, \ldots, x_n$, the weighted harmonic mean is defined by

$$H(x_1, x_2, \ldots, x_n) = \frac{\sum_{i=1}^{n} w_i}{\sum_{i=1}^{n} \frac{w_i}{x_i}}$$

6. Model Evaluation and Selection

6.1. Metrics for Evaluating Classifier Performance

- The **accuracy** measure works best when the data classes are fairly evenly distributed.
- Other measures, such as **sensitivity** (or **recall**), **specificity**, **precision**, ***F***, and F_β, are better suited to the class imbalance problem, where the main class of interest is rare.

6. Model Evaluation and Selection
6.2. Holdout Method and Random Subsampling

- In the **holdout method**, the given data are randomly partitioned into two independent sets, a *training set* and a *test set*.
- Typically, two-thirds of the data are allocated to the training set, and the remaining one-third is allocated to the test set. The training set is used to derive the model.
- The model's accuracy is then estimated with the test set. The estimate is pessimistic because only a portion of the initial data is used to derive the model.

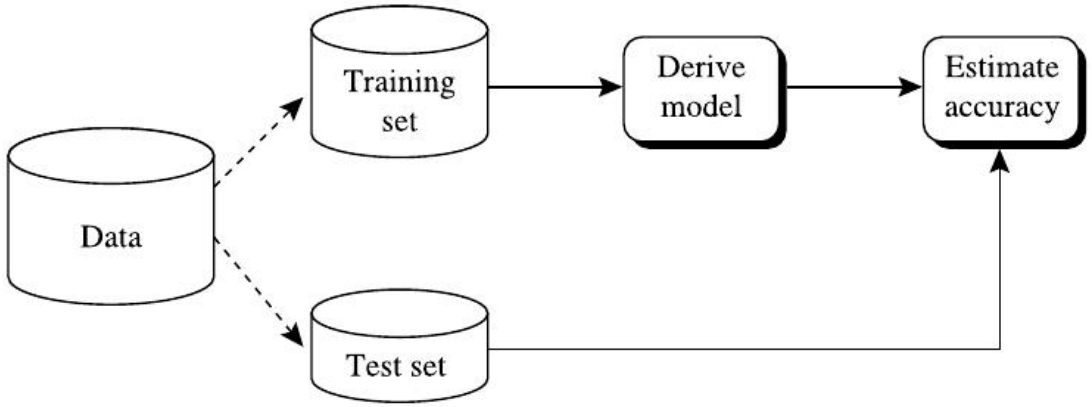


FIGURE 6.19

Estimating accuracy with the holdout method.

- **Random subsampling** is a variation of the holdout method in which the holdout method is repeated *k* times. The overall accuracy estimate is taken as the average of the accuracies obtained from each iteration.

6. Model Evaluation and Selection
6.3. Cross-Validation

- ***k*-fold cross-validation**,
  - The initial data are randomly partitioned into k mutually exclusive subsets or "folds" $D_1, D_2, ..., D_k$, each of approximately equal size.
  - Training and testing are performed k times. In iteration i, partition D_i is reserved as the test set, and the remaining partitions are collectively used to train the model.
  - Unlike the holdout and random subsampling methods, here each sample is used the same number of times for training and once for testing.
  - For classification, the accuracy estimate is the overall number of correct classifications from the k iterations, divided by the total number of tuples in the initial data.

6. Model Evaluation and Selection
6.3. Cross-Validation

- **Leave-one-out-cross-validation**
  - A special case of *k*-fold cross-validation where *k* is set to the number of initial tuples. That is, only one sample is "left out" at a time for the test set.
  - Leave-one-out-cross-validation is often used when the initial data set is small.
- In **stratified cross-validation**, the folds are stratified so that the **class distribution** of the tuples in each fold is approximately the same as that in the initial data.
- In practice, stratified 10-fold cross-validation is recommended for estimating accuracy (even if computation power allows using more folds) due to its relatively low bias and variance.

6. Model Evaluation and Selection
6.4. Bootstrap

- The **bootstrap method** samples the given training tuples uniformly with *replacement*.
- That is, each time a tuple is selected, it is equally likely to be selected again and re-added to the training set.
  - For instance, imagine a machine that randomly selects tuples for our training set. In *sampling with replacement*, the machine is allowed to select the same tuple more than once.
- There are several bootstrap methods. A commonly used one is the **.632** bootstrap, which works as follows:
  - Suppose we are given a data set of *d* tuples. The data set is sampled *d* times, with replacement, resulting in a bootstrap sample or training set of *d* samples.
  - Some of the original data tuples will likely occur more than once in this sample. The data tuples that did not make it into the training set end up forming the test set. Suppose we were to try this out several times. As it turns out, on average, 63.2% of the original data tuples will end up in the bootstrap sample, and the remaining 36.8% will form the test set (hence, the name, .632 bootstrap).

6. Model Evaluation and Selection
6.4. Bootstrap

- "*Where does the figure, 63.2%, come from?*"
- Each tuple has a probability of $1/d$ of being selected, so the probability of not being chosen is $(1 - 1/d)$.
- We have to select d times, so the probability that a tuple will not be chosen during this whole time is $(1 - 1/d)^d$.
- If d is large, the probability approaches $e^{-1} = 0.368.14$.
- Thus 36.8% of tuples will not be selected for training and thereby end up in the test set, and the remaining 63.2% will form the training set.

6. Model Evaluation and Selection

6.4. Bootstrap

- We can repeat the sampling procedure k times, wherein each iteration, we use the current test set to obtain an estimated accuracy of the model obtained from the current bootstrap sample.
- The overall accuracy of the model, M, is then estimated as

$$Acc(M) = \frac{1}{k}\sum_{i=1}^{k}(0.632 \times Acc(M_i)_{test\_set} + 0.368 \times Acc(M_i)_{train\_set}), \qquad (6.30)$$

where $Acc(M_i)_{test\_set}$ is the accuracy of the model obtained with bootstrap sample i when it is applied to test set i. $Acc(M_i)_{train\_set}$ is the accuracy of the model obtained with bootstrap sample i when it is applied to the original set of data tuples.

- Bootstrapping tends to be overly optimistic. It works best with small data sets.

6. Model Evaluation and Selection

6.5. Comparing Classifiers Based on Cost-Benefit and ROC Curves

- The cost associated with a false negative (such as incorrectly predicting that a cancerous patient is not cancerous) is far greater than those of a false positive (incorrectly yet conservatively labeling a noncancerous patient as cancerous).
  - In such cases, we can outweigh one type of error over another by assigning a different cost to each. These costs may consider the danger to the patient, financial costs of resulting therapies, and other hospital costs.
- Similarly, the benefits associated with a true positive decision may be different from those of a true negative.
- Up to now, to compute the classifier's accuracy, we have assumed equal costs and essentially divided the sum of true positives and true negatives by the total number of test tuples.
- Alternatively, we can incorporate costs and benefits by computing the average cost (or benefit) per decision.

6. Model Evaluation and Selection

6.5. Comparing Classifiers Based on Cost-Benefit and ROC Curves

- **Receiver operating characteristic curves** are a useful visual tool for comparing two classification models.
- ROC curves come from signal detection theory that was developed during World War II for the analysis of radar images. A ROC curve for a given model shows the trade-off between the *true positive rate* (TPR) and the *false positive rate* (FPR).
- Given a test set and a model, TPR is the proportion of positive (or "yes") tuples that are correctly labeled by the model; FPR is the proportion of negative (or "no") tuples that are mislabeled as positive.

$$TPR = \frac{TP}{P} = sensitive$$

$$FPR = \frac{FP}{N} = 1 - specificity$$

- For a two-class problem, a ROC curve allows us to visualize the trade-off between the rate at which the model can accurately recognize positive cases vs. the rate at which it mistakenly identifies negative cases as positive for different portions of the test set. Any increase in TPR occurs at the cost of an increase in FPR. The area under the ROC curve is a measure of the accuracy of the model.

6. Model Evaluation and Selection

6.5. Comparing Classifiers Based on Cost-Benefit and ROC Curves

- To plot a ROC curve for a given classification model, M, the model must be able to return a probability of the predicted class for each test tuple.
- With this information, we rank and sort the tuples so that the tuple that is most likely to belong to the positive or "yes" class appears at the top of the list, and the tuple that is least likely to belong to the positive class lands at the bottom of the list.
- Naïve Bayesian and logistic regression classifiers return a class probability distribution for each prediction and, therefore, are appropriate, although other classifiers, such as decision tree classifiers can easily be modified to return class probability predictions.
- Let the value that a probabilistic classifier returns for a given tuple $\boldsymbol{X}$ be $f(\boldsymbol{X}) \rightarrow [0, 1]$.
- For a binary problem, a threshold t is typically selected so that tuples where $f(\boldsymbol{X}) \geq t$ are considered positive and all the other tuples are considered negative. Note that the number of true positives and the number of false positives are both functions of t, so that we could write $TP(t)$ and $FP(t)$. Both are monotonic nonincreasing functions.

6. Model Evaluation and Selection

6.5. Comparing Classifiers Based on Cost-Benefit and ROC Curves

- The vertical axis of a ROC curve represents TPR. The horizontal axis represents FPR. To plot a ROC curve for M, we begin as follows.
  - Starting at the bottom left corner (where $TPR = FPR = 0$), we check the tuple's actual class label at the top of the list.
  - If we have a true positive (i.e., a positive tuple that was correctly classified), then TP and thus TPR increase. On the graph, we move up and plot a point.
  - If, instead, the model classifies a negative tuple as positive, we have a false positive, and so both FP and FPR increase. On the graph, we move right and plot a point.
  - This process is repeated for each of the test tuples in ranked order, each time moving up on the graph for a true positive or toward the right for a false positive.

6. Model Evaluation and Selection
6.5. Comparing Classifiers Based on Cost-Benefit and ROC Curves

- **Example 6.11. Plotting a ROC curve.**
  - 10 tuples in a test set, sorted in the decreasing probability order, $P = 5$, $N = 5$.
  - Start with tuple 1, which has the highest probability score, and take that score as our threshold, that is, $t = 0.9$.
    - The classifier considers tuple 1 to be positive, and all the other tuples are considered negative. --> TPR = 0.2, FPR = 0.
  - Next, threshold $t = 0.8$, the probability value for tuple 2
    - So this tuple 2 is now also considered positive, whereas tuples 3 through 10 are considered negative. --> TPR = 0.4, FPR = 0.2.

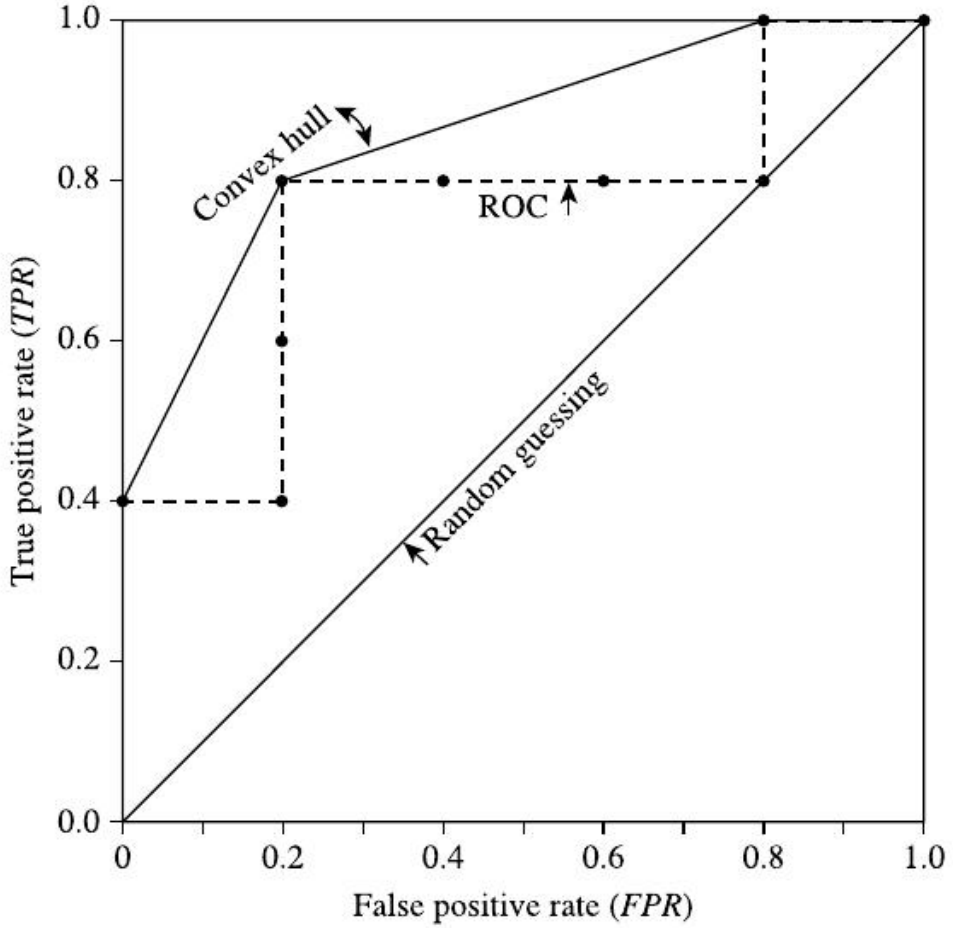
Tuple #	Class	Prob.	*TP*	*FP*	*TN*	*FN*	*TPR*	*FPR*
1	*P*	0.90	1	0	5	4	0.2	0
2	*P*	0.80	2	0	5	3	0.4	0
3	*N*	0.70	2	1	4	3	0.4	0.2
4	*P*	0.60	3	1	4	2	0.6	0.2
5	*P*	0.55	4	1	4	1	0.8	0.2
6	*N*	0.54	4	2	3	1	0.8	0.4
7	*N*	0.53	4	3	2	1	0.8	0.6
8	*N*	0.51	4	4	1	1	0.8	0.8
9	*P*	0.50	5	4	1	0	1.0	0.8
10	*N*	0.40	5	5	0	0	1.0	1.0

FIGURE 6.20

Tuples sorted by decreasing score, where the score is the value returned by a probabilistic classifier.

6. Model Evaluation and Selection

6.5. Comparing Classifiers Based on Cost-Benefit and ROC Curves



There are many methods to obtain a curve out of these points, the most common of which is to use a convex hull. The plot also shows a diagonal line where for every true positive of such a model, we are just as likely to encounter a false positive. For comparison, this line represents random guessing.

Tuple #	Class	Prob.	*TP*	*FP*	*TN*	*FN*	*TPR*	*FPR*
1	*P*	0.90	1	0	5	4	0.2	0
2	*P*	0.80	2	0	5	3	0.4	0
3	*N*	0.70	2	1	4	3	0.4	0.2
4	*P*	0.60	3	1	4	2	0.6	0.2
5	*P*	0.55	4	1	4	1	0.8	0.2
6	*N*	0.54	4	2	3	1	0.8	0.4
7	*N*	0.53	4	3	2	1	0.8	0.6
8	*N*	0.51	4	4	1	1	0.8	0.8
9	*P*	0.50	5	4	1	0	1.0	0.8
10	*N*	0.40	5	5	0	0	1.0	1.0

FIGURE 6.20

Tuples sorted by decreasing score, where the score is the value returned by a probabilistic classifier.

6. Model Evaluation and Selection

6.5. Comparing Classifiers Based on Cost-Benefit and ROC Curves

- The closer the ROC curve of a model is to the diagonal line, the less accurate the model.
- If the model is really good, initially we are more likely to encounter true positives as we move down the ranked list. Thus the curve moves steeply up from zero.
- Later, as we start to encounter fewer and fewer true positives, and more and more false positives, the curve eases off and becomes more horizontal.
- To assess the accuracy of a model, we can measure the area under the curve (AUC). Several software packages are able to perform such calculation. The closer the area is to 0.5, the less accurate the corresponding model is. A model with perfect accuracy will have an AUC of 1.0.

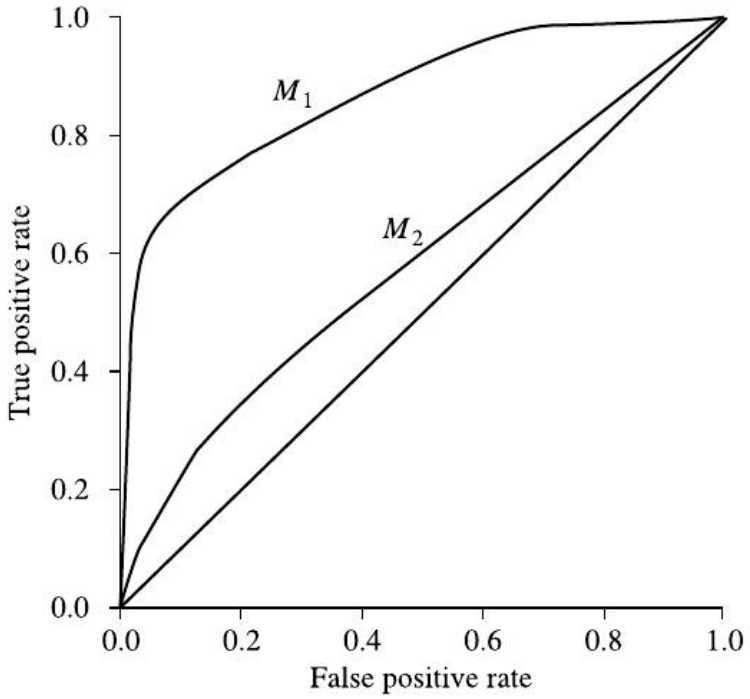


FIGURE 6.22

ROC curves of two classification models, M_1 and M_2. The diagonal shows where, for every true positive, we are equally likely to encounter a false positive. The closer a ROC curve is to the diagonal line, the less accurate the model is. Thus $M1$ is more accurate here.

7. Techniques to Improve Classification Accuracy
7.1. Introducing Ensemble Methods

- *Bagging*, *boosting*, and *random forests* are examples of **ensemble methods**.
- An ensemble combines a series of k learned models (or *base classifiers*), $M_1, M_2, ..., M_k$, with the aim of creating an improved composite classification model, M^*.
- A given data set, D, is used to create k training sets, $D_1, D_2, ..., D_k$, where D_i $(1 \leq i \leq k)$ is used to generate classifier M_i.
- Given a new data tuple to classify, the base classifiers each vote by returning a class prediction. The ensemble returns a class prediction based on the votes of the base classifiers.

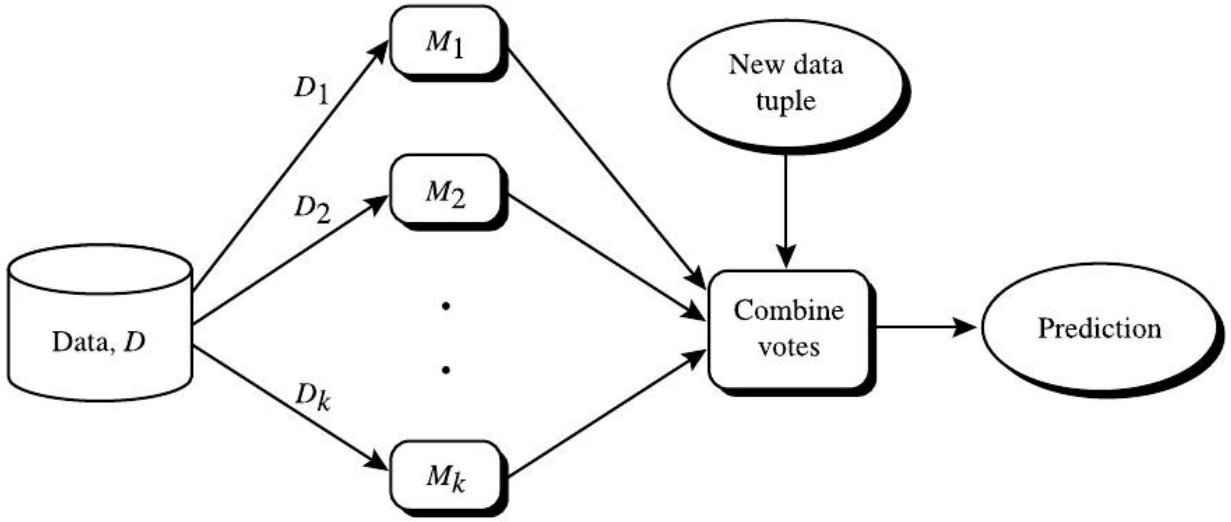


FIGURE 6.23

Increasing classifier accuracy. Ensemble methods generate a set of classification models, $M_1, M_2, \ldots, M_k$. Given a new data tuple to classify, each classifier "votes" for the class label of that tuple. The ensemble combines the votes to return a class prediction.

7. Techniques to Improve Classification Accuracy
7.1. Introducing Ensemble Methods

- An ensemble tends to be more accurate than its base classifiers.
  - The base classifiers may make mistakes, but the ensemble will misclassify ***X*** only if over half of the base classifiers are in error. Ensembles yield better results when there is significant diversity among the models.

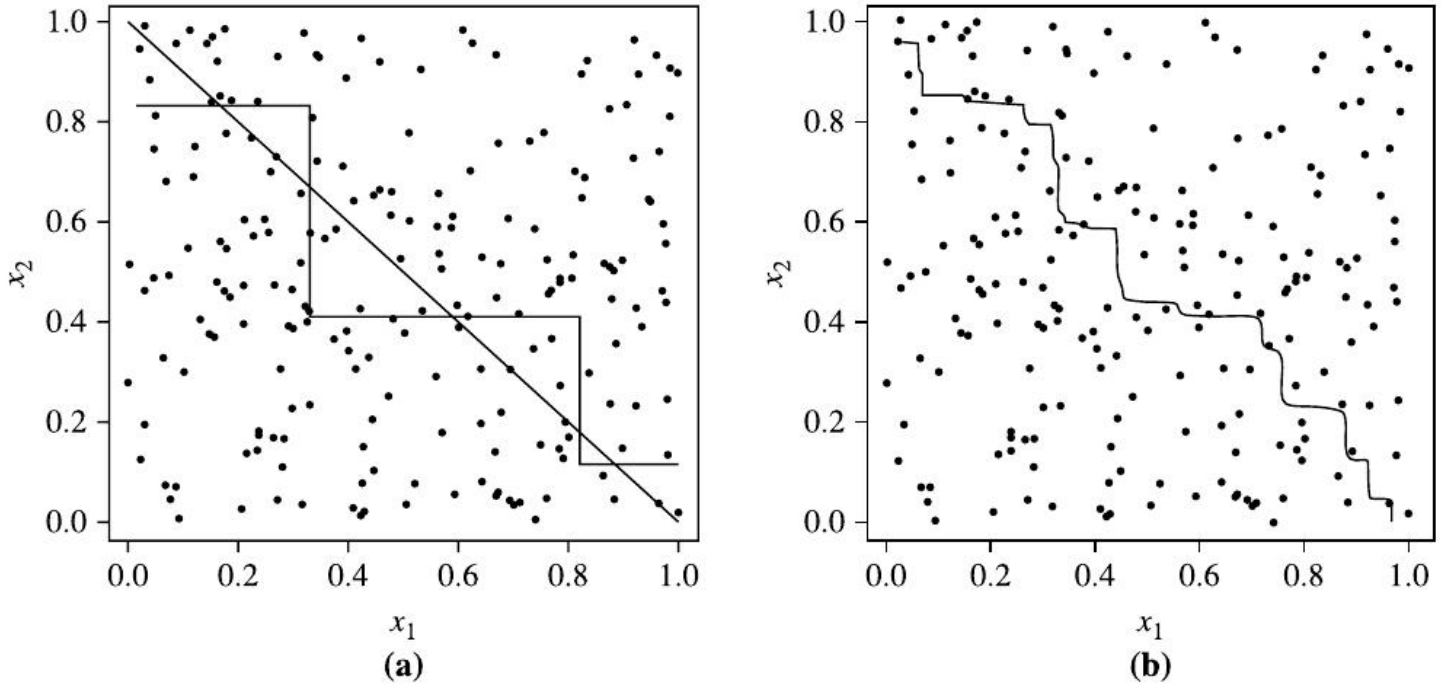


FIGURE 6.24

Decision boundary by (a) a single decision tree and (b) an ensemble of decision trees for a linearly separable problem (i.e., where the actual decision boundary is a straight line). The decision tree struggles with approximating a linear boundary. The decision boundary of the ensemble is closer to the true boundary. *Source:* From Seni and Elder [SE10]. © 2010 Morgan & Claypool Publishers; used with permission.

7. Techniques to Improve Classification Accuracy

7.2. Bagging

- We now take an intuitive look at how bagging works as a method of increasing accuracy.
- Suppose that you are a patient and would like to have a diagnosis made based on your symptoms.
  - Instead of asking one doctor, you may choose to ask several.
  - If a certain diagnosis occurs more than any other, you may choose this as the final or best diagnosis. That is, the final diagnosis is made based on a majority vote, where each doctor gets an equal vote.
- Now replace each doctor by a classifier, and you have the basic idea behind bagging. Intuitively, a majority vote made by a large group of doctors may be more reliable than a majority vote made by a small group.

7. Techniques to Improve Classification Accuracy
7.2. Bagging

- Given a set, D, of d tuples, **bagging** works as follows. For iteration i ($i = 1, 2, ..., k$), a training set, D_i, of d tuples is sampled with replacement from the original set of tuples, D.
  - Note that the term *bagging* stands for *bootstrap aggregation*. Each training set is a bootstrap sample.
- Because sampling with replacement is used, some of the original tuples of D may not be included in D_i, whereas others may occur more than once.
- A classifier model, M_i, is learned for each training set, D_i.
- To classify an unknown tuple, $\boldsymbol{X}$, each classifier, M_i, returns its class prediction, which counts as one vote.
- The bagged classifier, M^*, counts the votes and assigns the class with the most votes to $\boldsymbol{X}$.
- Bagging can be applied to the prediction of continuous values by taking the average value of each prediction for a given test tuple.

7. Techniques to Improve Classification Accuracy
7.2. Bagging

Algorithm: Bagging. The bagging algorithm—create an ensemble of classification models for a learning scheme where each model gives an equally weighted prediction.

Input:

- D, a set of d training tuples;
- k, the number of models in the ensemble;
- a classification learning scheme (e.g., decision tree algorithm, naïve Bayesian, etc.).

Output: The ensemble—a composite model, M^*.

Method:

(1) **for** $i = 1$ to k **do** // create k models:
(2) create bootstrap sample, D_i, by sampling D with replacement;
(3) use D_i and the learning scheme to derive a model, M_i.
(4) **endfor**

To use the ensemble to classify a tuple, X:

let each of the k models classify $\boldsymbol{X}$ and return the majority vote;

7. Techniques to Improve Classification Accuracy
7.2. Bagging

- The bagged classifier often has significantly greater accuracy than a single classifier derived from *D*, the original training data.
- It is often more robust to the effects of noisy data and overfitting. The increased accuracy occurs because the composite model reduces the variance of the individual classifiers.

7. Techniques to Improve Classification Accuracy
7.3. Boosting

- As in the previous section, suppose that as a patient, you have certain symptoms. Instead of consulting one doctor, you choose to consult several. Suppose **you assign weights to the value or worth** of each doctor's diagnosis based on the accuracies of previous diagnoses they have made. The final diagnosis is then a combination of the weighted diagnoses. This is the essence behind boosting.

- In **boosting**, weights are also assigned to each training tuple. A series of k classifiers is iteratively learned. After a classifier, M_i, is learned, the weights are updated to allow the subsequent classifier, M_{i+1}, to "pay more attention" to the training tuples that were misclassified by M_i. The final boosted classifier, M^*, combines the votes of each individual classifier, where **the weight of each classifier's vote is a function of its accuracy.**

7. Techniques to Improve Classification Accuracy

7.3. Boosting

Adaboost

- **AdaBoost** (short for Adaptive Boosting) is a popular boosting algorithm.
- Initially, AdaBoost assigns each training tuple an equal weight of $1/d$. Generating k classifiers for the ensemble requires k rounds through the rest of the algorithm. We can sample to form any sized training set, not necessarily of size d. Sampling with replacement is used—the same tuple may be selected more than once.
- **Each tuple's chance of being selected is based on its weight.** A classifier model, M_i, is derived from the training tuples of D_i. Its error is then calculated using D as the test set. The weights of the tuples are then adjusted according to how they were classified.
  - If a tuple was incorrectly classified, its weight is increased. If a tuple was correctly classified, its weight is decreased. A tuple's weight reflects how difficult it is to classify—the higher the weight, the more often it has been misclassified.
  - These weights will be used to generate the training samples for the classifier of the next round. The basic idea is that when we build a classifier, we want it to focus more on the misclassified tuples of the previous round. Some classifiers may be better at classifying some "difficult" tuples than others. In this way, we build a series of classifiers that complement each other.

7. Techniques to Improve Classification Accuracy

7.3. Boosting

Adaboost

Algorithm: AdaBoost. A boosting algorithm—create an ensemble of classifiers. Each one gives a weighted vote.

Input:

- D, a set of d class-labeled training tuples;
- k, the number of rounds (one classifier is generated per round);
- a classification learning scheme.

Output: A composite model.
Method:

(1) initialize the weight of each tuple in D to $1/d$;
(2) **for** $i = 1$ to k **do** // for each round:
(3) sample D with replacement according to the tuple weights to obtain D_i;
(4) use training set D_i to derive a model, M_i;
(5) compute $error(M_i)$, the error rate of M_i (Eq. (6.34))
(6) **if** $error(M_i) > 0.5$ **then**
(7) abort the loop;
(8) **endif**
(9) **for** each tuple in D that was correctly classified **do**
(10) multiply the weight of the tuple by $error(M_i)/(1 - error(M_i))$; // update weights
(11) normalize the weight of each tuple.
(12) **endfor**

7. Techniques to Improve Classification Accuracy

7.3. Boosting

Adaboost

To use the ensemble to classify tuple, X:

(1) initialize weight of each class to 0;
(2) **for** $i = 1$ to k **do** // for each classifier:
(3) $w_i = log \frac{1-error(M_i)}{error(M_i)}$; // weight of the classifier's vote
(4) $c = M_i(\boldsymbol{X})$; // get class prediction for $\boldsymbol{X}$ from M_i
(5) add w_i to the weight for class c
(6) **endfor**
(7) return the class with the largest weight.

FIGURE 6.26

AdaBoost, a boosting algorithm.

7. Techniques to Improve Classification Accuracy

7.3. Boosting

Adaboost

- To compute the error rate of model M_i, we sum the weights of each of the tuples in D that M_i misclassified. That is,

$$error(M_i) = \sum_{j=1}^{d} w_j \times err(\mathbf{X}_j), \qquad (6.34)$$

where $err(\mathbf{X}_j)$ is the misclassification error of tuple $\mathbf{X}_j$: If the tuple was misclassified, then $err(\mathbf{X}_j)$ is 1; otherwise, it is 0.

- If the performance of classifier M_i is so poor that its error exceeds 0.5, then we abandon it. Instead, we try again by generating a new D_i training set, from which we derive a new M_i.

- The error rate of M_i affects how the weights of the training tuples are updated. If a tuple in the round i was correctly classified, its weight is multiplied by $error(M_i)/(1 - error(M_i))$. Once the weights of all the correctly classified tuples are updated, the weights for all tuples (including the misclassified ones) are normalized so that their sum remains the same as it was before. To normalize a weight, we multiply it by the sum of the old weights, divided by the sum of the new weights. As a result, the weights of misclassified tuples are increased, and the weights of correctly classified tuples are decreased, as described before.

7. Techniques to Improve Classification Accuracy

7.3. Boosting

Adaboost

- "*Once boosting is complete, how is the ensemble of classifiers used to predict the class label of a tuple, X?*"
- Unlike bagging, where each classifier was assigned an equal vote, boosting assigns a weight to each classifier's vote, based on how well the classifier performed. The lower a classifier's error rate, the more accurate it is, and therefore, the higher its weight for voting should be.
- The weight of classifier M_i 's vote is

$$log \frac{1 - error(M_i)}{error(M_i)}. \qquad (6.35)$$

- For each class, c, we sum the weights of each classifier that assigned class c to $\boldsymbol{X}$. The class with the highest sum is the "winner" and is returned as the class prediction for tuple $\boldsymbol{X}$.

7. Techniques to Improve Classification Accuracy

7.3. Boosting

- "*How does boosting compare with bagging?*"
- Because of the way boosting focuses on the misclassified tuples, it risks overfitting the resulting composite model to such data. Therefore sometimes the resulting "boosted" model may be less accurate than a single model derived from the same data.
- Bagging is less susceptible to model overfitting. While both can significantly improve accuracy in comparison to a single model, boosting tends to achieve greater accuracy.

7. Techniques to Improve Classification Accuracy
7.4. Random Forests

- Imagine that each of the classifiers in the ensemble is a *decision tree* classifier so that the collection of classifiers is a "forest."
- The individual decision trees are generated using a random selection of attributes at each node to determine the split.
- More formally, each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest.
- During classification, each tree votes, and the most popular class is returned.
- Random forests can be built using bagging.
- A training set, D, of d tuples is given. The general procedure to generate k decision trees for the ensemble is as follows. For each iteration, i ($i = 1, 2, \ldots, k$), a training set, D_i, of d tuples is sampled with replacement from D.
  - That is, each D_i is a bootstrap sample of D, so that some tuples may occur more than once in D_i, while others may be excluded.
  - Let F be the number of attributes to be used to determine the split at each node, where F is much smaller than the number of available attributes. To construct a decision tree classifier, M_i, randomly select, at each node, F attributes as candidates for the split at the node. The CART methodology is used to grow the trees. The trees are grown to maximum size and are not pruned. Random forests formed this way, with *random input selection*, are called Forest-RI.

7. Techniques to Improve Classification Accuracy
7.4. Random Forests

- Another form of random forest, called Forest-RC, uses *random linear combinations* of the input attributes.
- Instead of randomly selecting a subset of the attributes, it creates new attributes (or features) that are a linear combination of the existing attributes.
  - That is, an attribute is generated by specifying L, the number of original attributes to be combined.
  - At a given node, L attributes are randomly selected and added together with coefficients that are uniform random numbers on $[-1, 1]$. F linear combinations are generated, and a search is made over these for the best split.
  - This form of random forest is useful when there are only a few attributes available, so as to reduce the correlation between individual classifiers.